

A Promising Semantics for Relaxed-Memory Concurrency

Seoul National University
(Korea)



Jeehoon Kang
Chung-Kil Hur

MPI-SWS
(Germany)



MAX-PLANCK-GESELLSCHAFT

Ori Lahav
Viktor Vafeiadis
Derek Dreyer

A Message Passing Example: No Data Race

Initially: $D = F = 0$

D = 42

LOCK(L)

F = 1

UNLOCK(L)

while (1) {

LOCK(L)

f = F

UNLOCK(L)

if (f) break

}

d = D

Finally: $d = 42$

Sequentially Consistent Concurrency

Initially: $D = F = 0$

D = 42

F = 1

while (1) {

f = F

if (f) break

}

d = D

Finally: $d = 42$

Relaxed-Memory Concurrency

Initially: $D = F = 0$

F = 1 **D = 42**

 **HW out of order exec**

D = 42 **F = 1**

```
while (1) {  
    f = F  
  
    if (f) break  
}  
  
d = D
```

Finally: $d = 42$ or 0

Release & Acquire

Initially: $D = F = 0$

$D = 42$

$F = 1$ [rel]

while (1) {

$f = F$ [acq]

if (f) **break**

}

$d = D$

Finally: $d = 42$

Release & Acquire

Initially: $D = F = 0$

$D = 42$

$F = 1$ [rel]

Run as if
in a single thread

while (1) {

$f = F$ [acq]

if (f) break

}

$d = D$

Finally: $d = 42$

Release & Acquire

Initially: $D = F = 0$

$D = 42$

$F = 1$ [rel]

Run as if
in a single thread

while (1) {

$f = F$ [acq]

if (f) break

}

$d = D$

$D = 42$

$F = 1$

Finally: $d = 42$ or $d = 0$

“Right” Concurrency Semantics?

Conflicting goals of compilers, hardware & programmers

- **Compiler/hardware:** validating optimizations (e.g. reordering, merging)
- **Programmer:** supporting reasoning principles (e.g. DRF theorem, program logic)

"Right" Concurrency Semantics?

Conflicting goals of compilers, hardware & programmers



Java memory model

- **Compiler/hardware:** validating optimizations (e.g. reordering, merging)
- **Programmer:** supporting reasoning principles (e.g. DRF theorem, program logic)

"Right" Concurrency Semantics?

Conflicting goals of compilers, hardware & programmers



Java memory model

- **Compiler/hardware:** validating optimizations (e.g. reordering, merging)
- **Programmer:** supporting reasoning principles (e.g. DRF theorem, program logic)



C/C++ memory model

“Right” Concurrency Semantics?

Conflicting goals of compilers, hardware & programmers



Java memory model

- **Compiler/hardware:** validating optimizations (e.g. reordering, merging)
- **Programmer:** supporting reasoning principles (e.g. DRF theorem, program logic)



C/C++ memory model

Key problem: “out-of-thin-air”

"Out-of-thin-air" problem (1/3)

Store Hoisting (= Load-Buffering)

Thread 1 **Thread 2**

$a = X$

$b = Y$

$Y = a$

$X = 42$

(allowed: $a=b=42$)

"Out-of-thin-air" problem (1/3)

Hoi = Load-Buffering

Registers

Shared
Locations

Thread 1

Thread 2

$a = X$

$b = Y$

$Y = a$

$X = 42$

(allowed: $a=b=42$)

"Out-of-thin-air" problem (1/3)

Store Hoisting (= Load-Buffering)

Thread 1	Thread 2
----------	----------

$a = X$	$b = Y$
---------	---------

$Y = a$	$X = 42$
---------	----------

(allowed: $a=b=42$)

"Out-of-thin-air" problem (1/3)

Store Hoisting (= Load-Buffering)

Thread 1 Thread 2

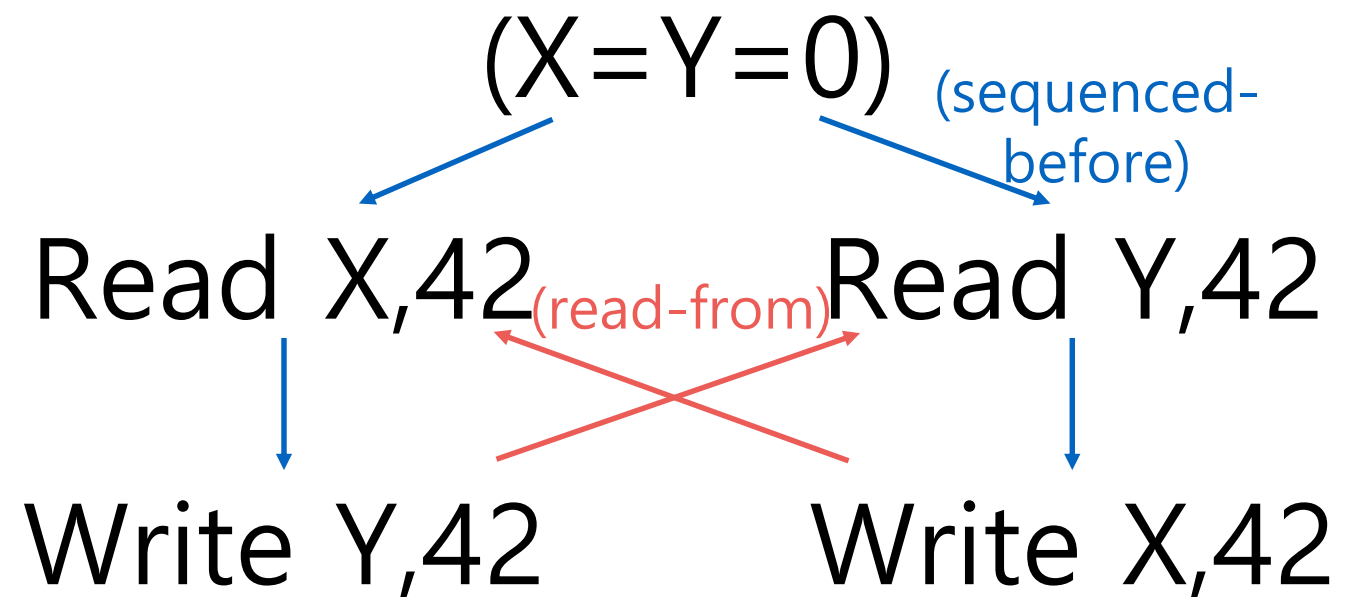
$a = X$

$b = Y$

$Y = a$

$X = 42$

(allowed: $a=b=42$)



"Out-of-thin-air" problem (1/3)

Store Hoisting (= Load-Buffering)

Thread 1 Thread 2

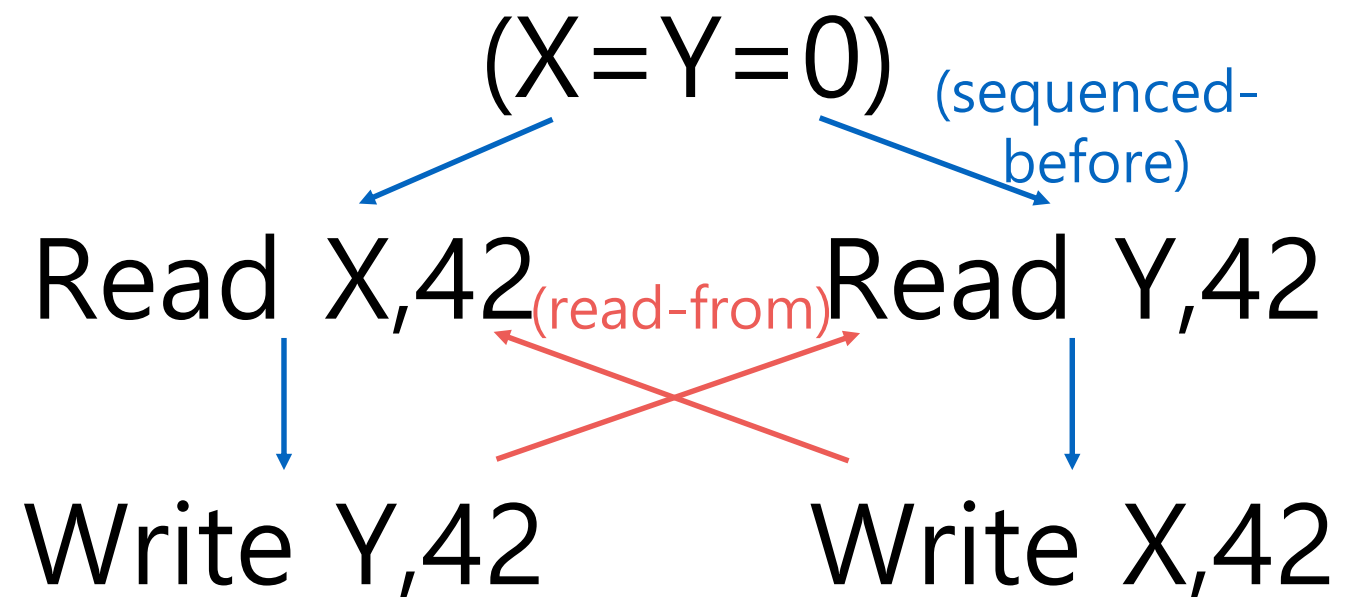
a = X

b = Y

Y = a

X = 42

(allowed: a=b=42)



Allowed by justification
(C/C++)

Justification is
too loose!

"Out-of-thin-air" problem (2/3)

Classical Out-of-thin-air (OOTA)

Thread 1	Thread 2
----------	----------

$a = X$	$b = Y$
---------	---------

$Y = a$	$X = b$
---------	---------

(forbidden: $a=b=42$)

"Out-of-thin-air" problem (2/3)

Classical Out-of-thin-air (OOTA)

Thread 1 **Thread 2**

$a = X$

$b = Y$

$Y = a$

$X = b$

(**forbidden**: $a=b=42$)

42 is out-of-thin-air!

Reasoning principles
(e.g. invariant $a=b=X=Y=0$)

"Out-of-thin-air" problem (2/3)

Classical Out-of-thin-air (OOTA)

Thread 1 Thread 2

$a = X$

$b = Y$

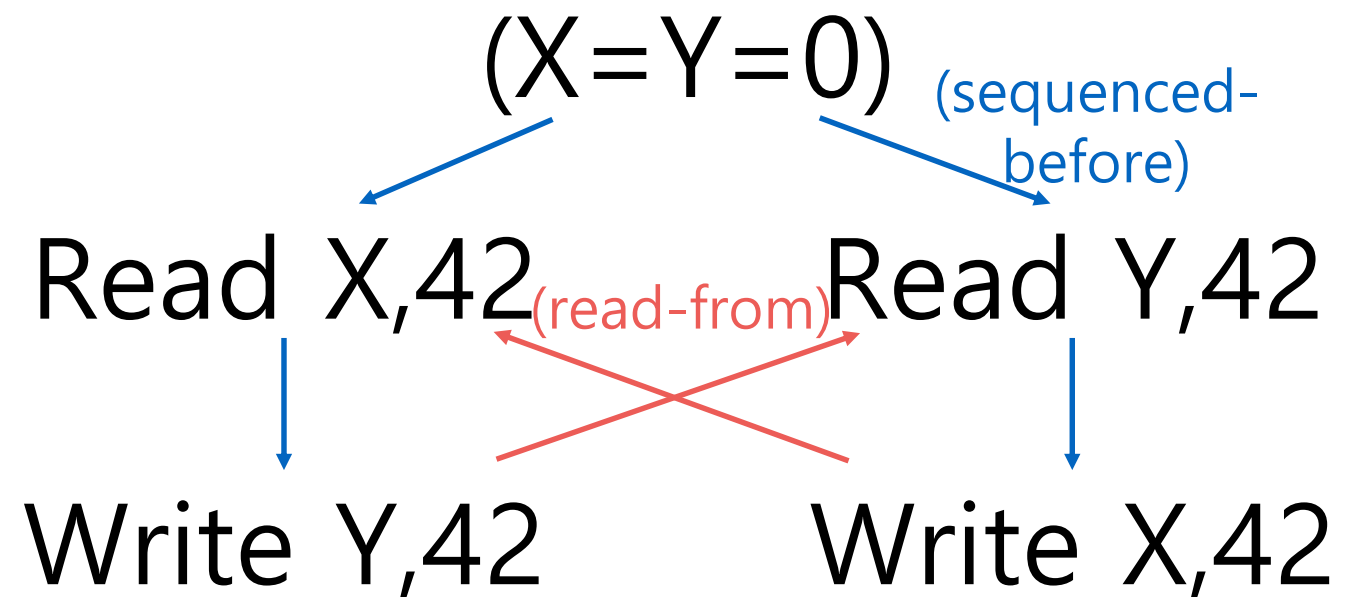
$Y = a$

$X = b$

(forbidden: $a=b=42$)

42 is out-of-thin-air!

Reasoning principles
(e.g. invariant $a=b=X=Y=0$)



Allowed by justification
(C/C++)

"Out-of-thin-air" problem (3/3)

Tweaked Store Hoisting

Thread 1

```
a = X  
Y = a
```

Thread 2

```
b = Y  
  
if (b==42) {  
    X = b; out(0)  
} else {  
    X = 42; out(1)  
}
```

(a=b=42, out(0) ?)

"Out-of-thin-air" problem (3/3)

Tweaked Store Hoisting

Thread 1

```
a = X  
Y = a
```

Thread 2

```
b = Y
```

```
if (b==42) {  
    X = b; out(0)  
} else {  
    X = 42; out(1)  
}
```



```
b = Y
```

```
X = 42
```

```
if (b==42) {  
    out(0)  
} else {  
    out(1)  
}
```

(a=b=42, out(0) ?)

"Out-of-thin-air" problem (3/3)

Tweaked Store Hoisting

Thread 1

```
a = X  
Y = a
```

Thread 2

```
b = Y  
c = X  
if (b==42) {  
    X = b; out(0)  
} else {  
    X = 42; out(1)  
}
```



```
b = Y  
c = X; X = 42  
if (b==42) {  
    out(0)  
} else {  
    out(1)  
}
```

(a=b=42, out(0) ?)

(c=42 ?)

“Out-of-thin-air” problem (3/3)

“A major open problem for PL semantics”
(Batty et al. ESOP 2015)

Thread 1

a = X
Y = a

Thread 2

b = Y
c = X
if (b==42) {
 X = b; out(0)
} else {
 X = 42; out(1)
}



b = Y
c = X; X = 42
if (b==42) {
 out(0)
} else {
 out(1)
}

(a=b=42, out(0) ?)

(c=42 ?)

Weak Behaviors (1/2)

Load Hoisting (= Store Buffering)

Thread 1

$Y = 42$

$a = X$

Thread 2

$X = 42$

$b = Y$

(allowed: $a=b=0$)

Weak Behaviors (1/2)

Load Hoisting (= Store Buffering)

Thread 1

Y = 42

a = X

(allowed: a=b=0)

Thread 2

X = 42

b = Y

 **reorderable**
(x86/Power/ARM)

b = Y

X = 42

Weak Behaviors (2/2)

Store Hoisting (= Load-Buffering)

Thread 1 **Thread 2**

$a = X$

$b = Y$

$Y = a$

$X = 42$

(allowed: $a=b=42$)

Weak Behaviors (2/2)

Store Hoisting (= Load-Buffering)

Thread 1 **Thread 2**

$a = X$

$b = Y$

$Y = a$

$X = 42$

(allowed: $a=b=42$)

 $X = 42$
reorderable
(Power/ARM) $b = Y$

Promising Semantics

- Solving the **out-of-thin-air problem**
- Supporting **optimizations & reasoning principles**
- Covering most **C/C++ concurrency** features
- **Operational semantics w/o undefined behavior**

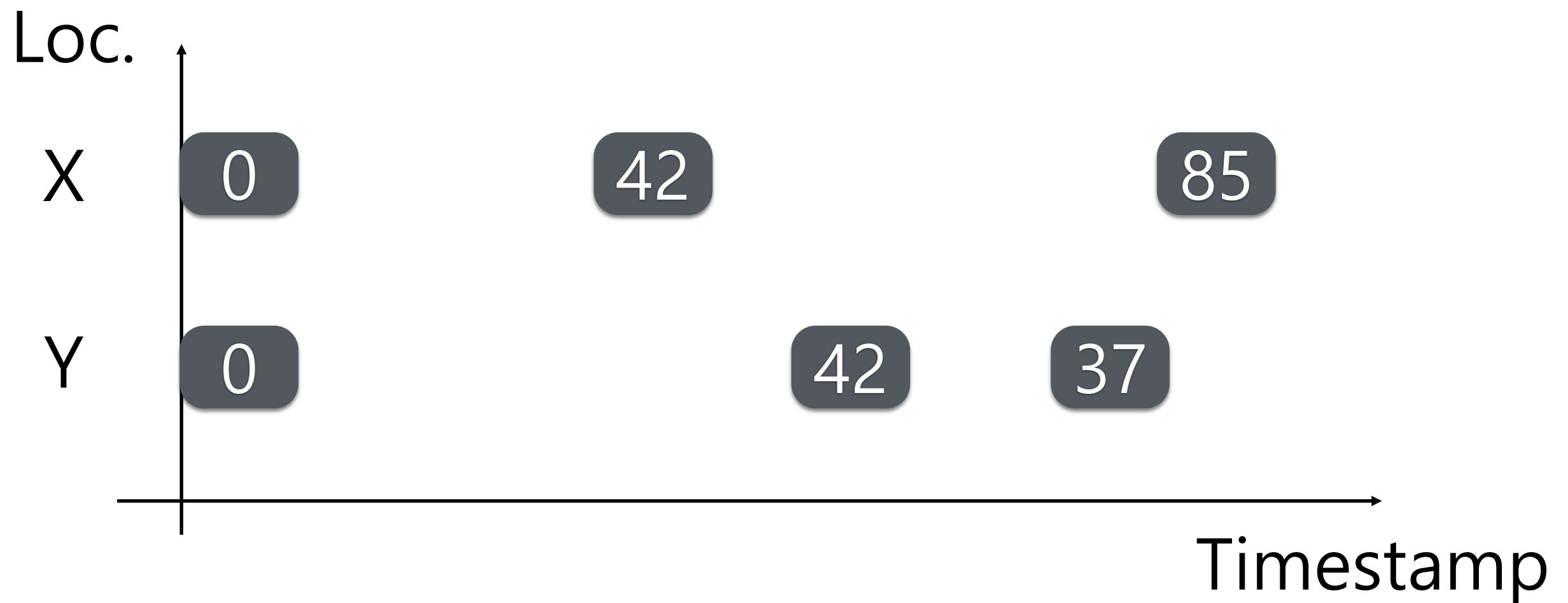
- Most results are **verified in Coq**



<http://sf.snu.ac.kr/promise-concurrency>

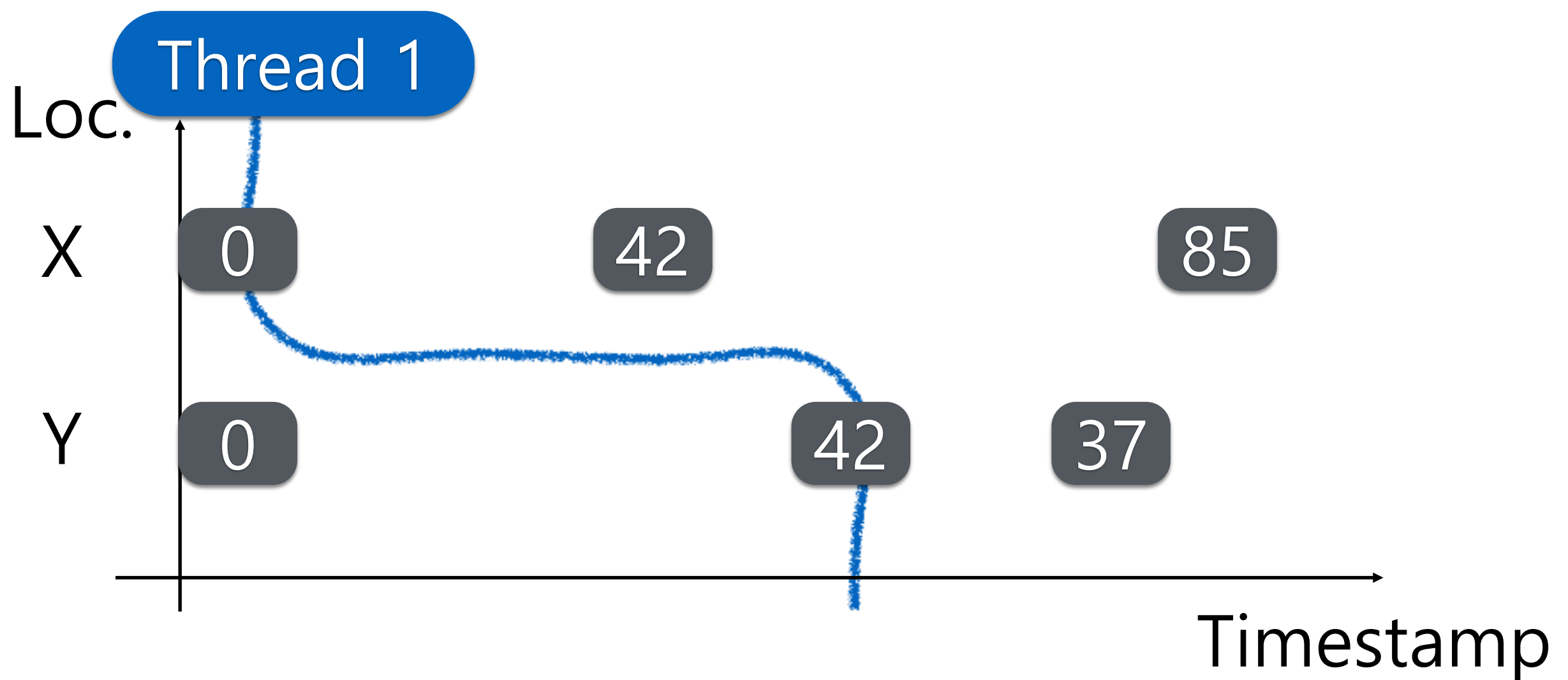
Key Idea 1: Messages & Views

- Memory: pool of **messages** (loc, val, timestamp)
- Per-thread **view** on the memory



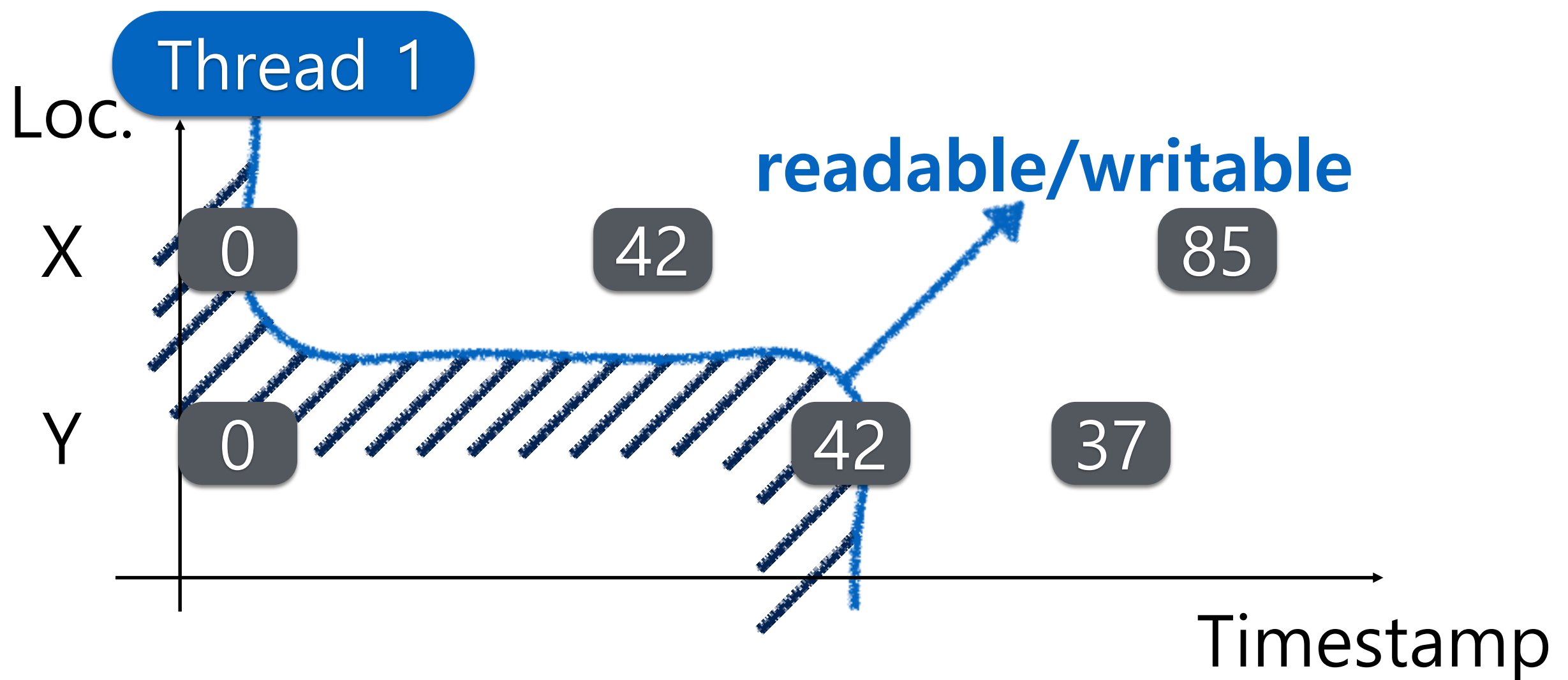
Key Idea 1: Messages & Views

- Memory: pool of **messages** (loc, val, timestamp)
- Per-thread **view** on the memory



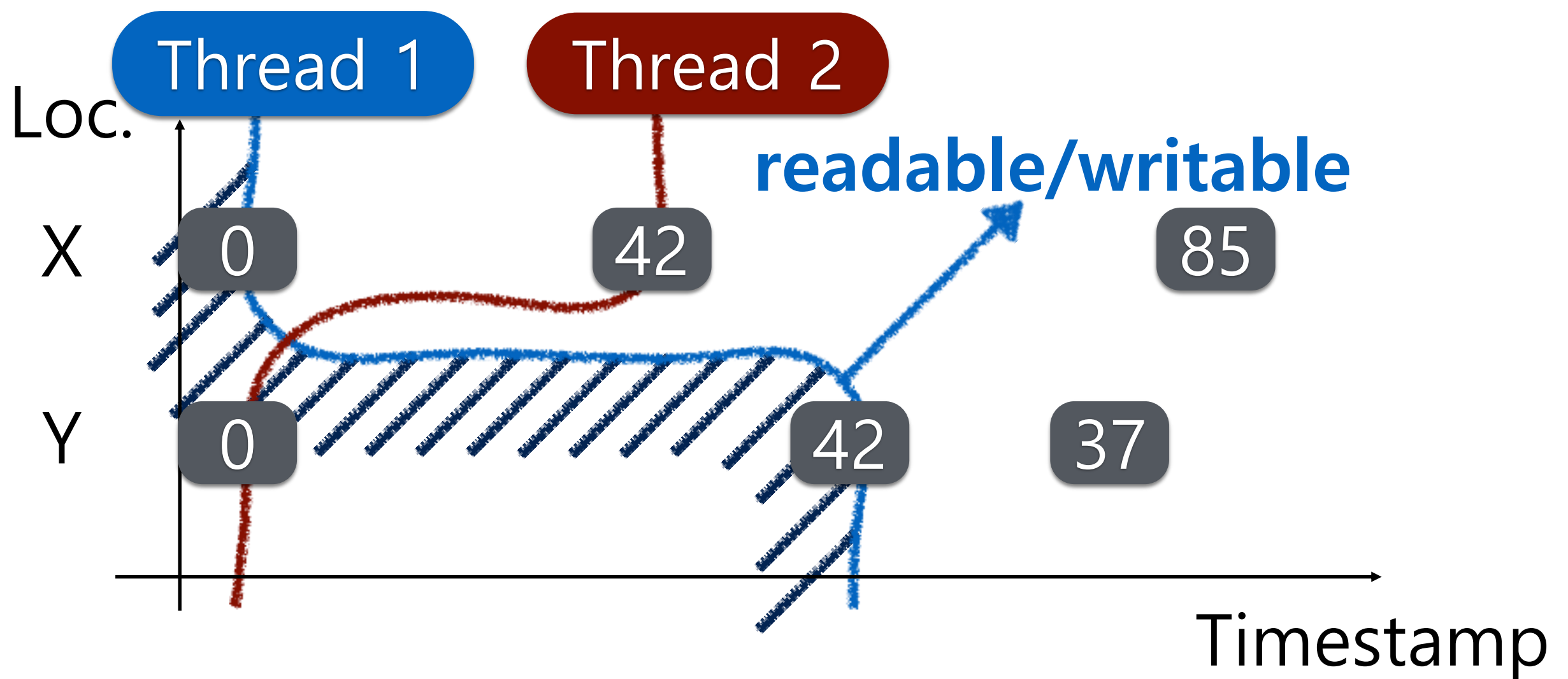
Key Idea 1: Messages & Views

- Memory: pool of **messages** (loc, val, timestamp)
- Per-thread **view** on the memory



Key Idea 1: Messages & Views

- Memory: pool of **messages** (loc, val, timestamp)
- Per-thread **view** on the memory



Example

Load Hoisting

Thread 1



$Y = 42$

$a = X$

Thread 2



$X = 42$

$b = Y$

(allowed: $a=b=0$)

Loc.

X

0

Y

0

Timestamp

Example Load Hoisting

Thread 1



$Y = 42$

$a = X$

Thread 2



$X = 42$

$b = Y$

(allowed: $a=b=0$)



reorderable

(x86/Power/ARM)

$b = Y$

$X = 42$

Loc.

X

0

Y

0

Timestamp

Example Load Hoisting

Thread 1



$Y = 42$

$a = X$

Thread 2



$X = 42$

$b = Y$

(allowed: $a=b=0$)

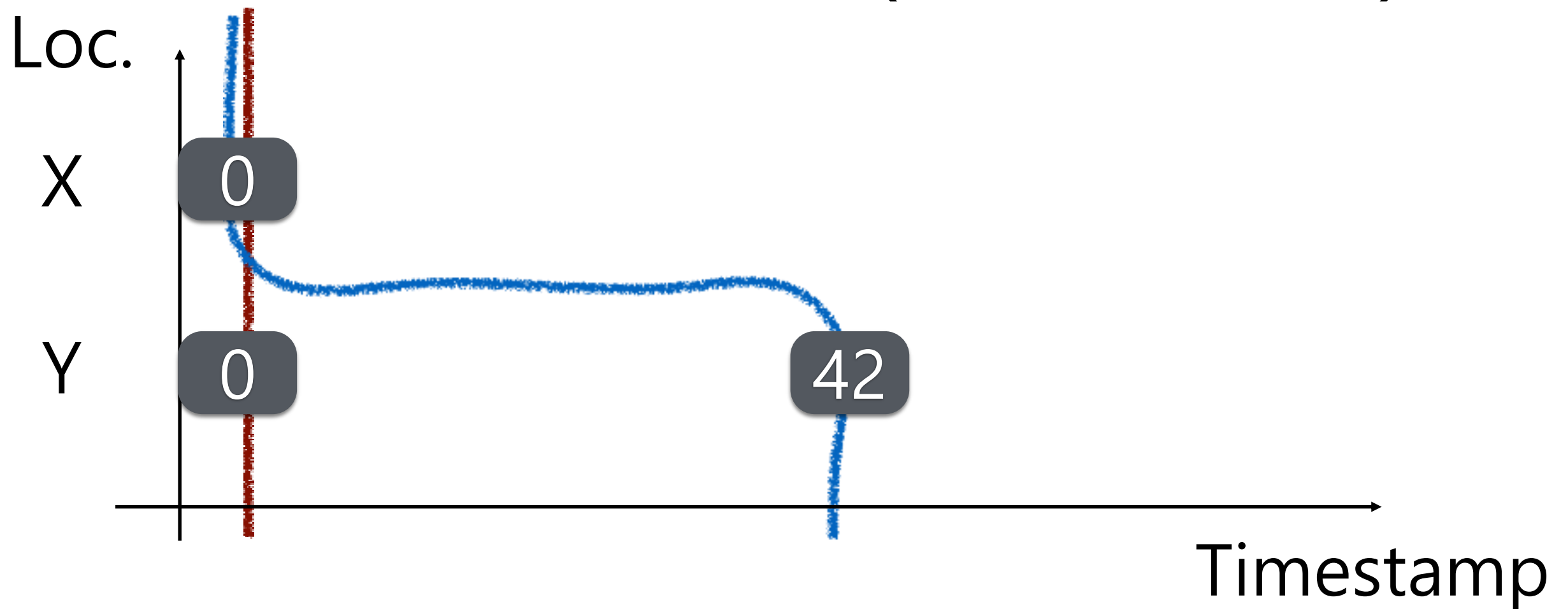


reorderable

(x86/Power/ARM)

$b = Y$

$X = 42$



Example Load Hoisting

Thread 1



$Y = 42$

$a = X$

Thread 2



$X = 42$

$b = Y$

(allowed: $a=b=0$)

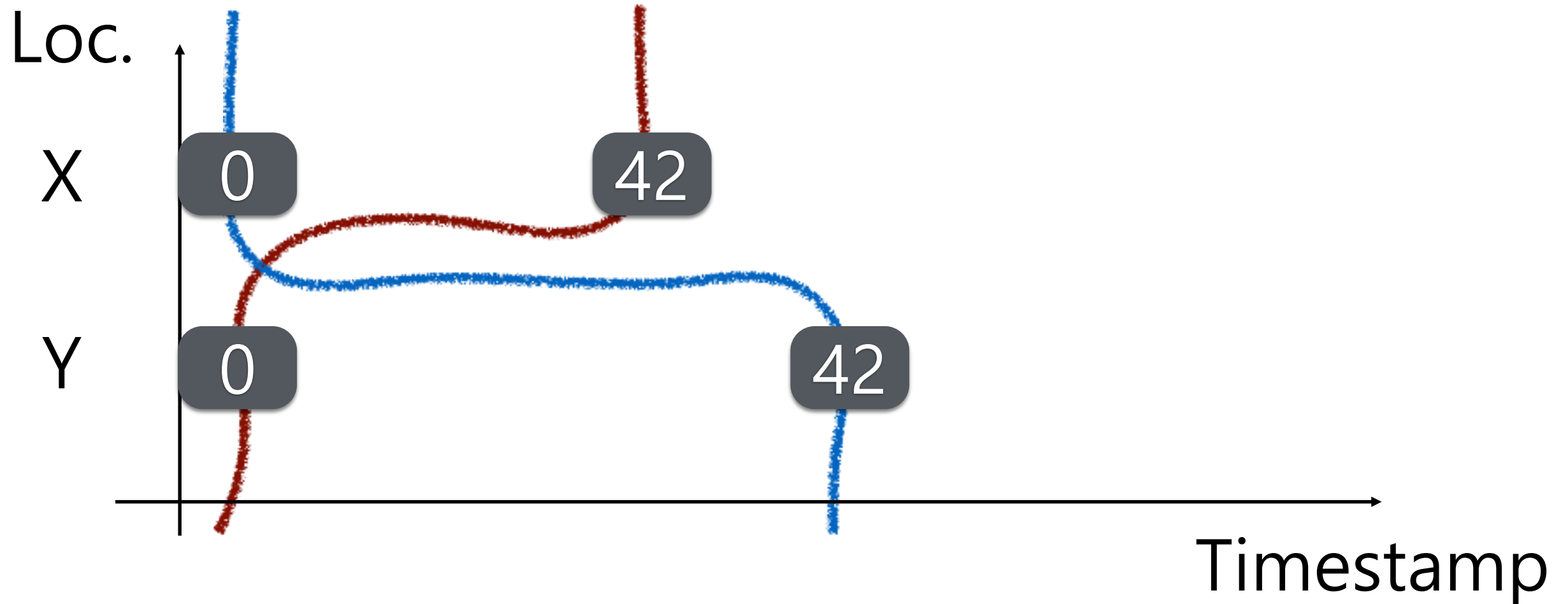


reorderable

(x86/Power/ARM)

$b = Y$

$X = 42$



Example Load Hoisting

Thread 1

$Y = 42$
 $a = X$

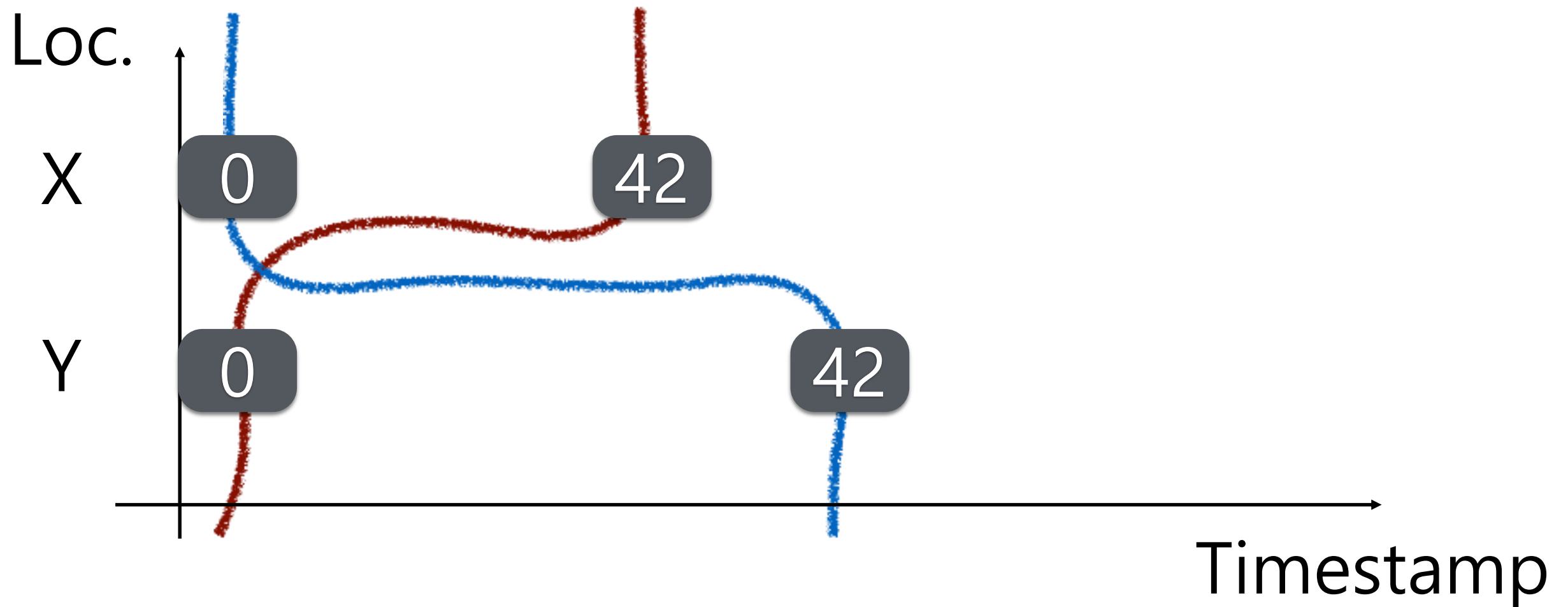
Thread 2

$X = 42$
 $b = Y$

(allowed: $a=b=0$)

reorderable
(x86/Power/ARM)

$b = Y$
 $X = 42$



Example Load Hoisting

Thread 1

$\forall$
0 = 42
a = X

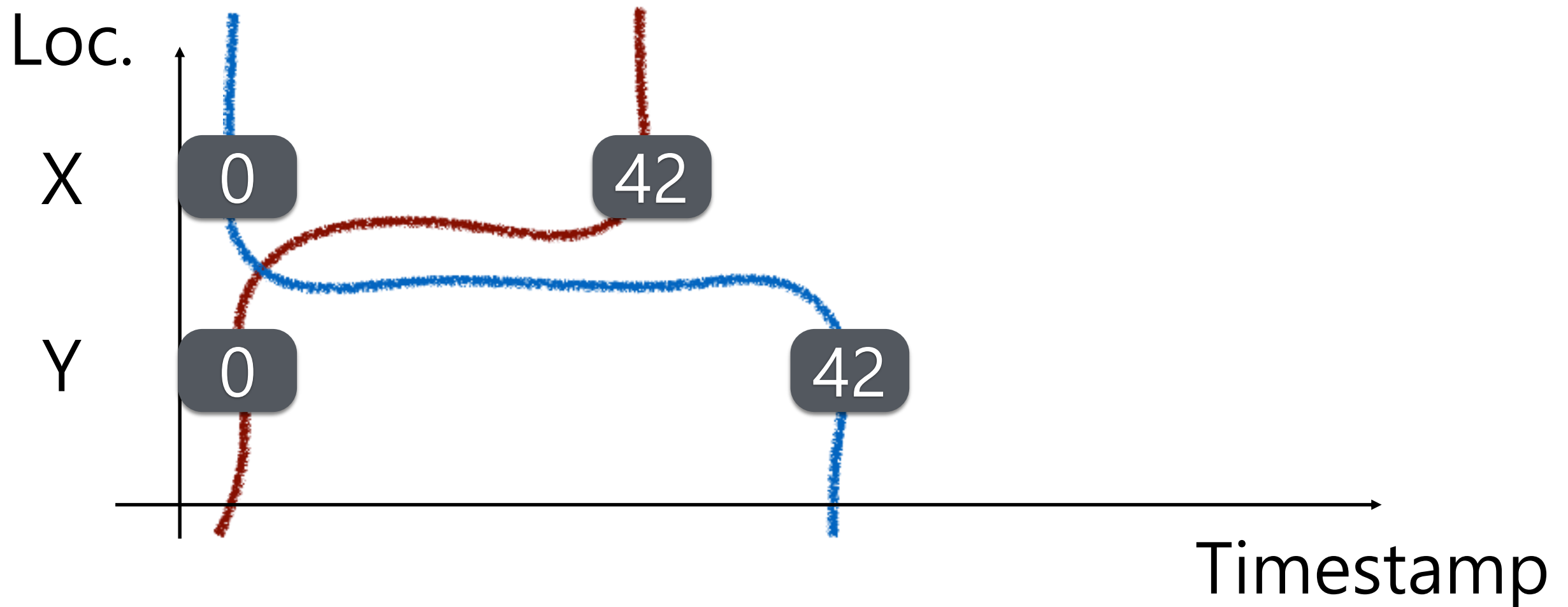
Thread 2

$\forall$
0 = 42
b = Y

(allowed: $a=b=0$)

reorderable
(x86/Power/ARM)

b = Y
X = 42



Example

Store Hoisting

Thread 1



$a = X$

$Y = a$

Thread 2



$b = Y$

$X = 42$

(allowed: $a=b=42$)

Loc.

X

0

Y

0

Timestamp

Key Idea 2: Promises

- A thread can **promise** to write $X=V$ in the future, after which **other threads can read** $X=V$.
- To avoid OOTA, the promising thread must **certify** that **whatever happens to the memory** it can write all its promises **in isolation**.
- **Until all its promises are fulfilled**, the thread can take only **certified steps**.

Example

Tweaked Store Hoisting

Thread 1



$a = X$

$Y = a$

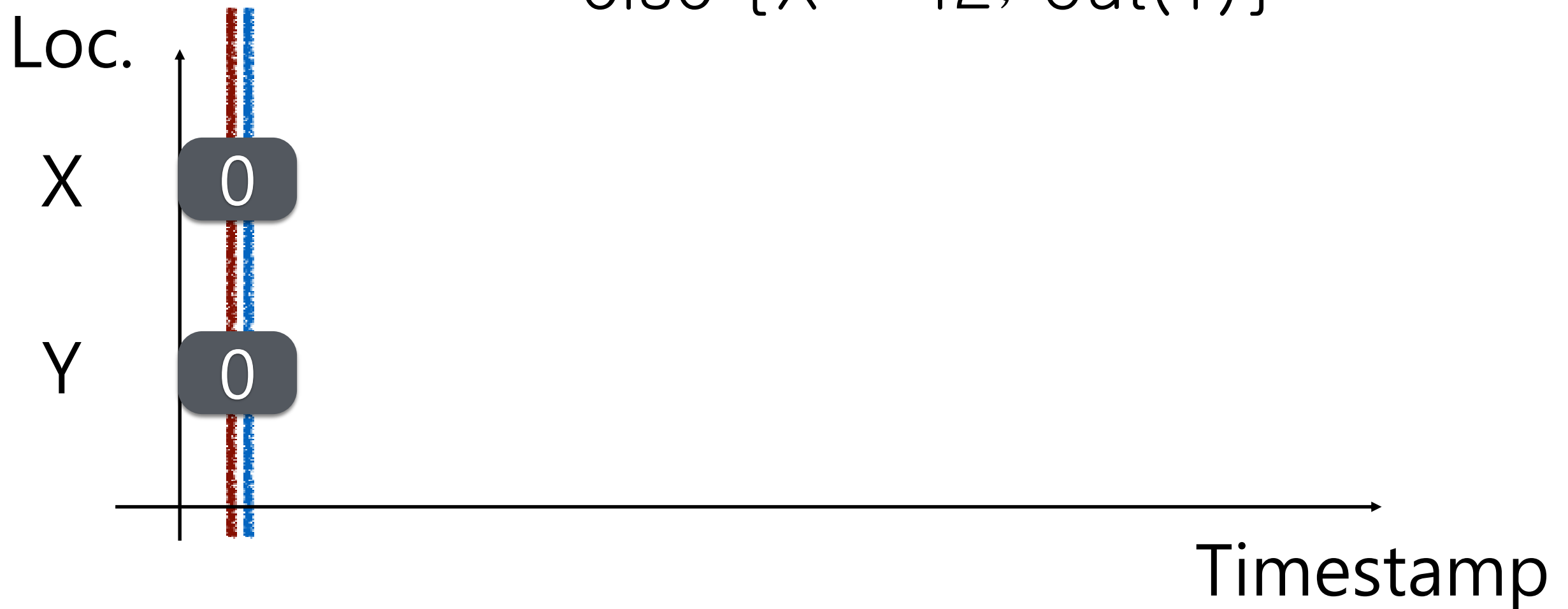
Thread 2 (allowed: $a=b=42, out(0)$)



$b = Y; c = X$

if ($b==42$) { $X = b; out(0)$ }

else { $X = 42; out(1)$ }



Example

Tweaked Store Hoisting

Thread 1



$a = X$

$Y = a$

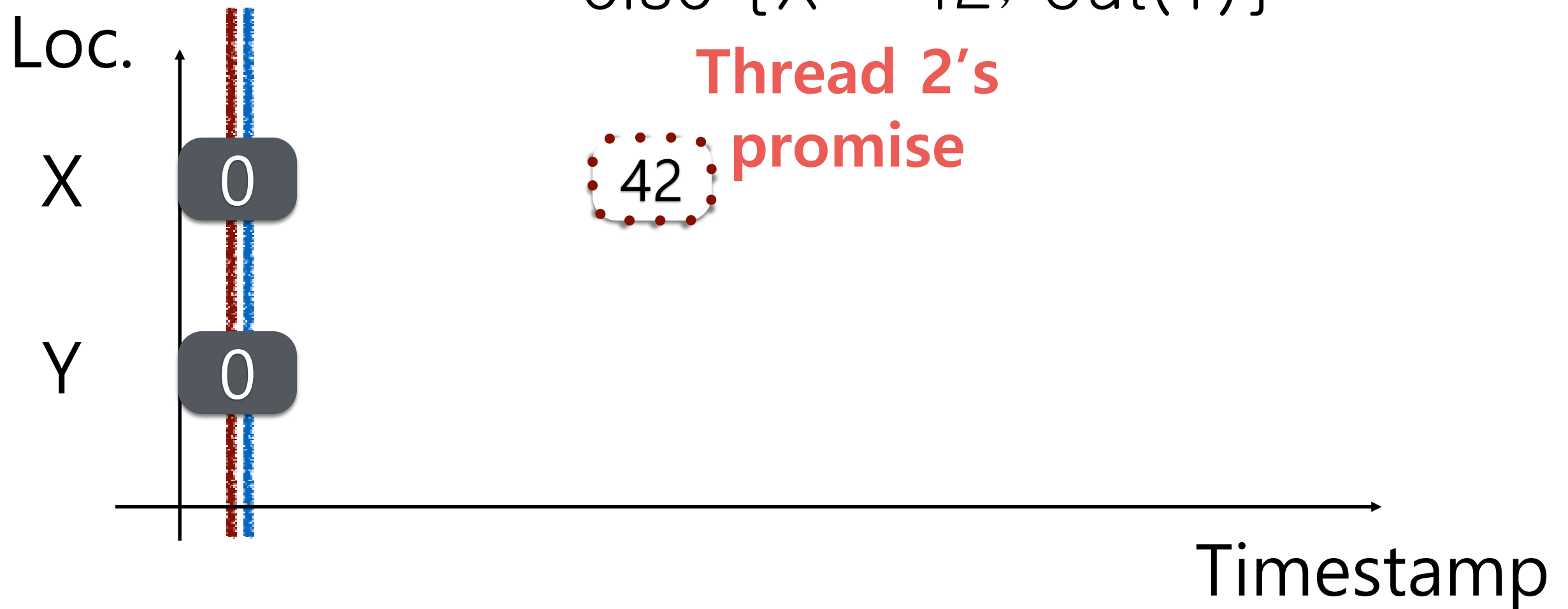
Thread 2 (allowed: $a=b=42, out(0)$)



$b = Y; c = X$

if ($b==42$) { $X = b; out(0)$ }

else { $X = 42; out(1)$ }



Example Tweaked Store Ho

Thread 2 should be
able to write it
in isolation

Thread 1

$a = X$

$Y = a$

Thread 2 (allowed: $a=b=42, out(0)$)

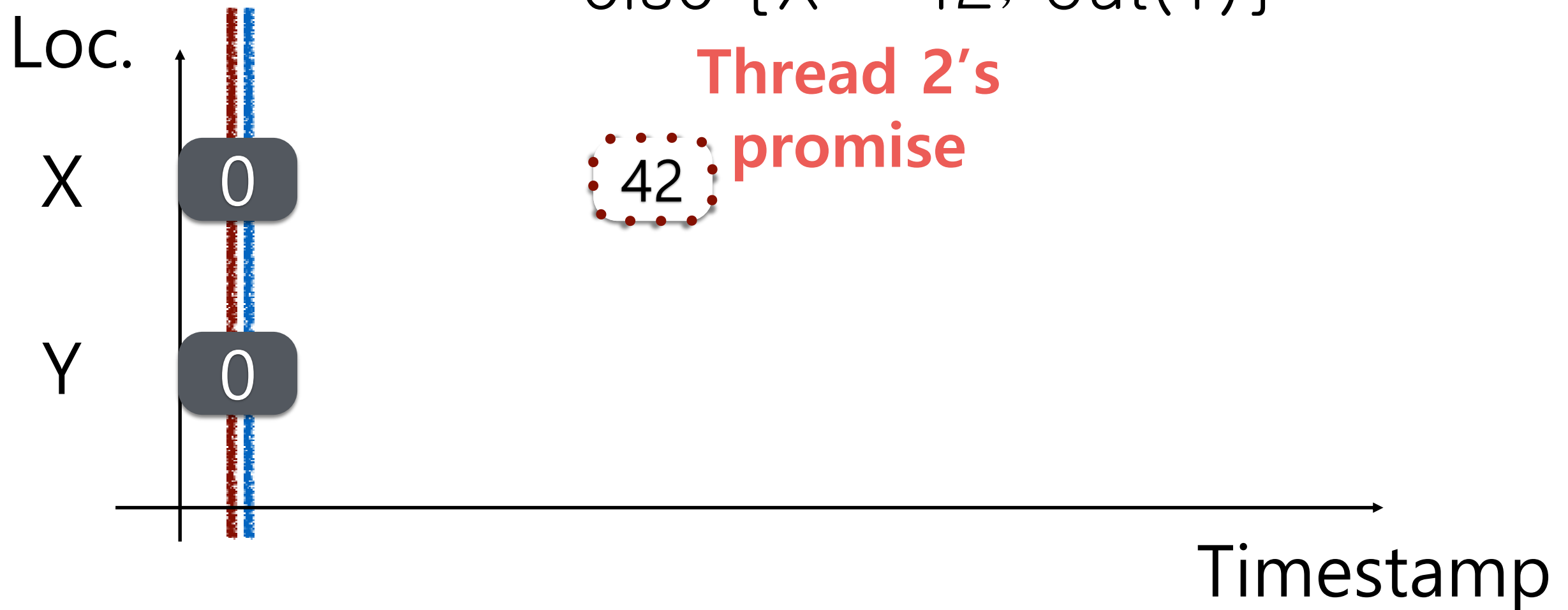
$b = Y; c = X$

if ($b==42$) { $X = b; out(0)$ }

else { $X = 42; out(1)$ }

Thread 2's

42 promise



Example Certification

Thread 2 should be
able to write it
in isolation

Thread 2



$b = Y; c = X$

if ($b == 42$) { $X = b; \text{out}(0)$ }

else { $X = 42; \text{out}(1)$ }

Thread 2's

42

promise



Example Certification

Thread 2 should be
able to write it
in isolation

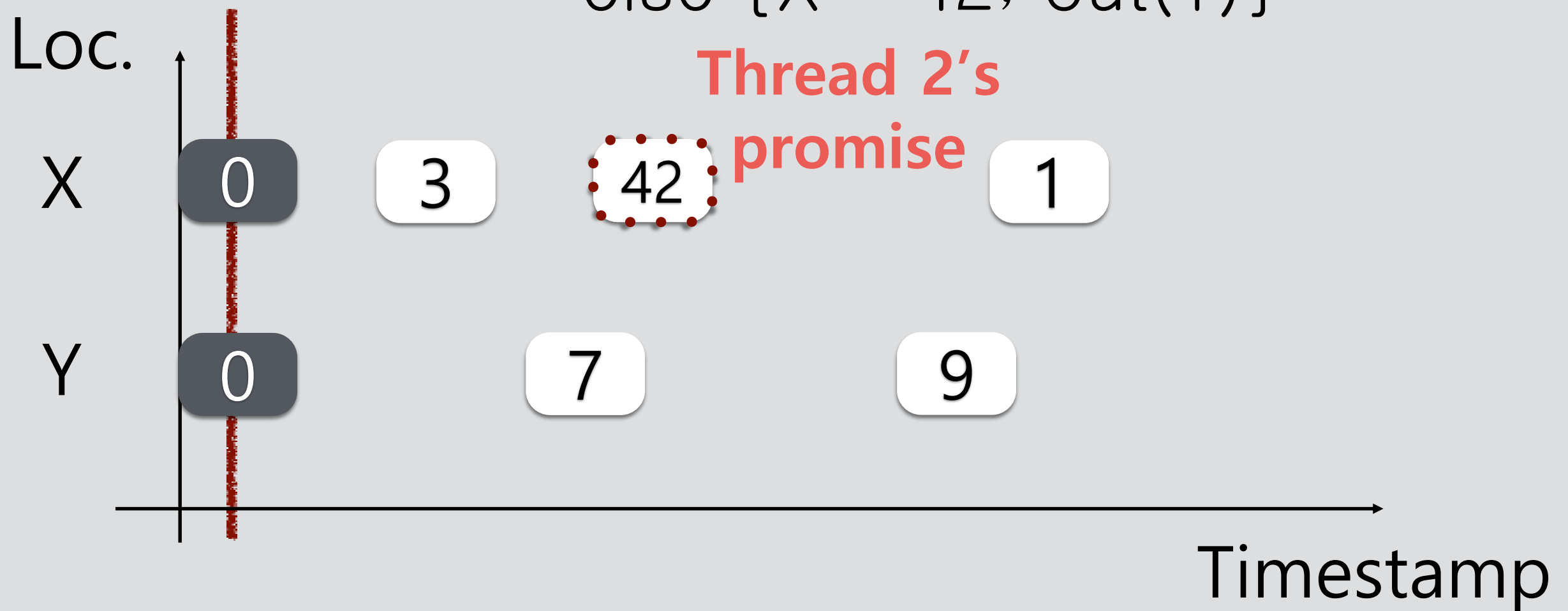
Thread 2



$b = Y; c = X$

if ($b == 42$) { $X = b; \text{out}(0)$ }

else { $X = 42; \text{out}(1)$ }



Example Certification

Thread 2 should be
able to write it
in isolation

Thread 2

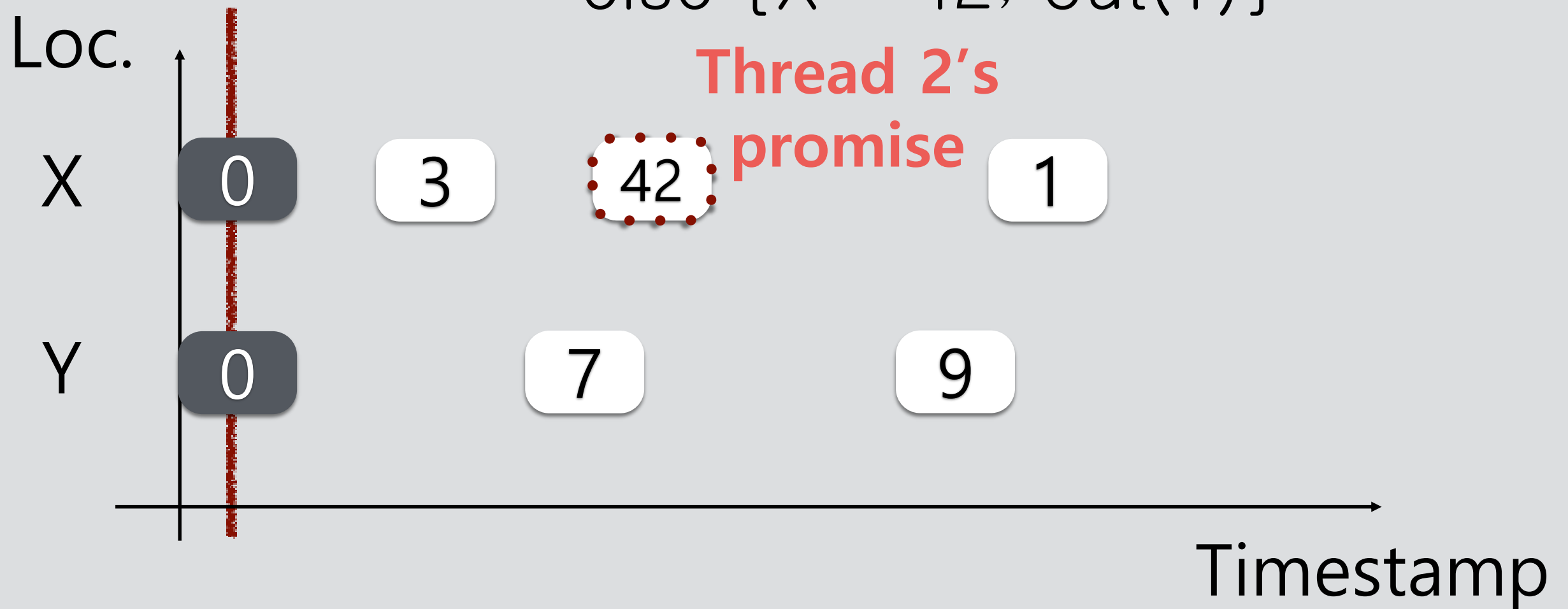
0

0

$b = Y; c = X$

if ($b == 42$) { $X = b; \text{out}(0)$ }

else { $X = 42; \text{out}(1)$ }



Example Certification

Thread 2 should be
able to write it
in isolation

Thread 2

0

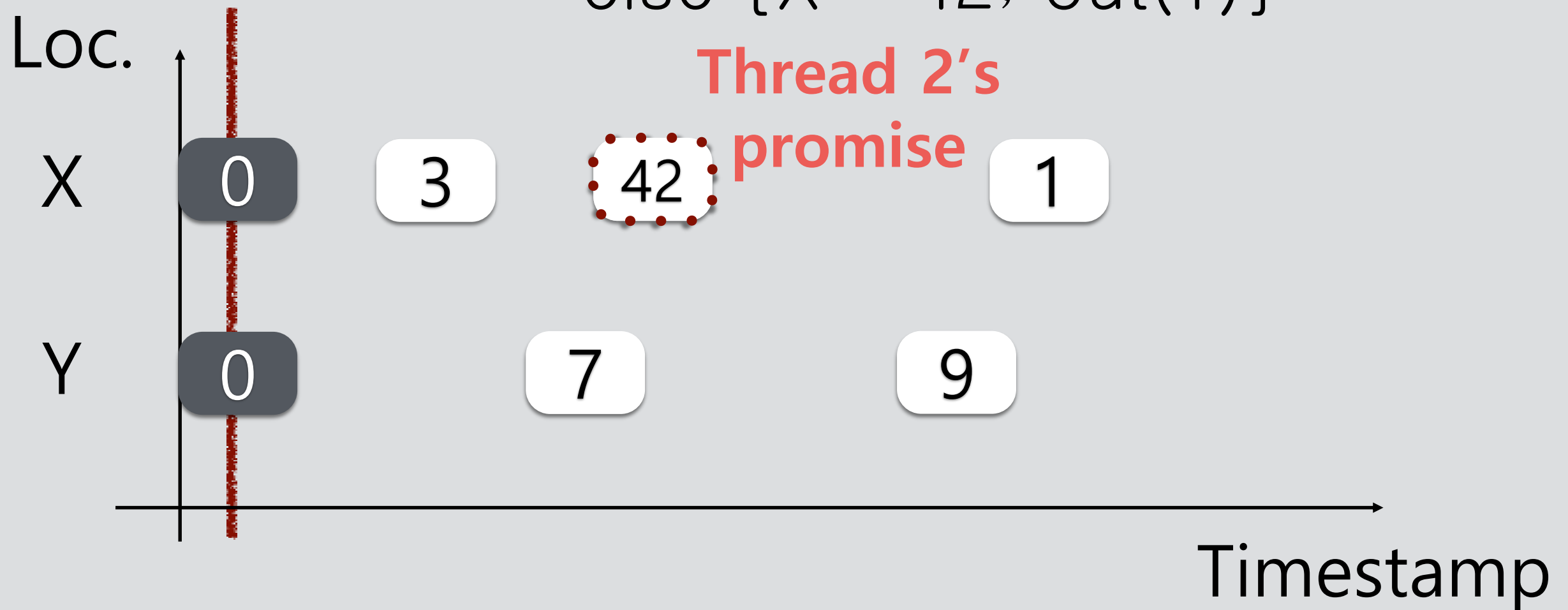
0

$b = Y; c = X$

if ($b == 42$) { $X = b$; out(0) }

else { $X = 42$; out(1) }

**Thread 2's
promise**



Example Certification

Thread 2 should be
able to write it
in isolation

Thread 2

0

0

$b = Y; c = X$

if ($b == 42$) { $X = b; \text{out}(0)$ }

else { $X = 42; \text{out}(1)$ }

**Thread 2's
promise**

Loc.

X

0

3

42

1

Y

0

7

9

Timestamp

Example Certification

Promise is
certified!

Thread 2

0

0

$b = Y; c = X$

if ($b == 42$) { $X = b; \text{out}(0)$ }

else { $X = 42; \text{out}(1)$ }

Loc.

X

0

3

42

1

Y

0

7

9

Thread 2's
promise

Timestamp

Example

Tweaked Store Hoisting

Thread 1



$a = X$

$Y = a$

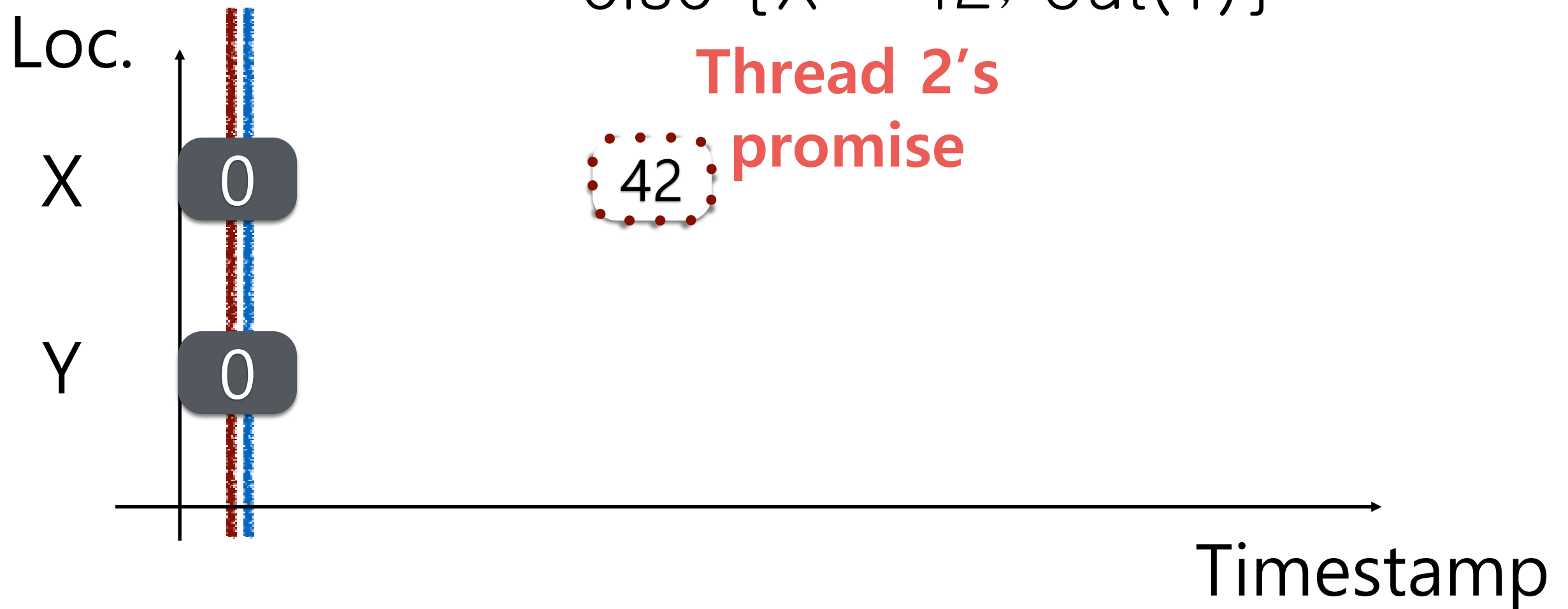
Thread 2 (allowed: $a=b=42, out(0)$)



$b = Y; c = X$

if ($b==42$) { $X = b; out(0)$ }

else { $X = 42; out(1)$ }



Example

Tweaked Store Hoisting

Thread 1

42

$a = X$

$Y = a$

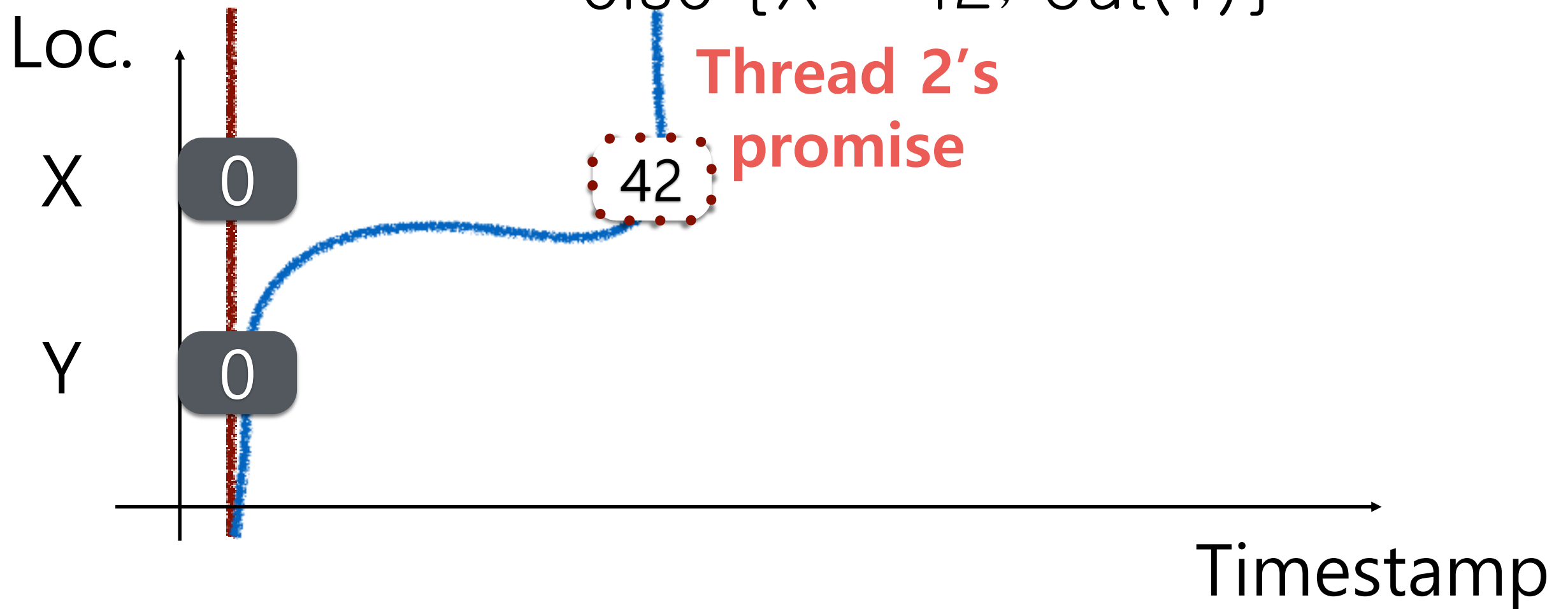
Thread 2 (allowed: $a=b=42, out(0)$)



$b = Y; c = X$

if ($b==42$) { $X = b; out(0)$ }

else { $X = 42; out(1)$ }



Example

Tweaked Store Hoisting

Thread 1

42

$a = X$

$Y = a$

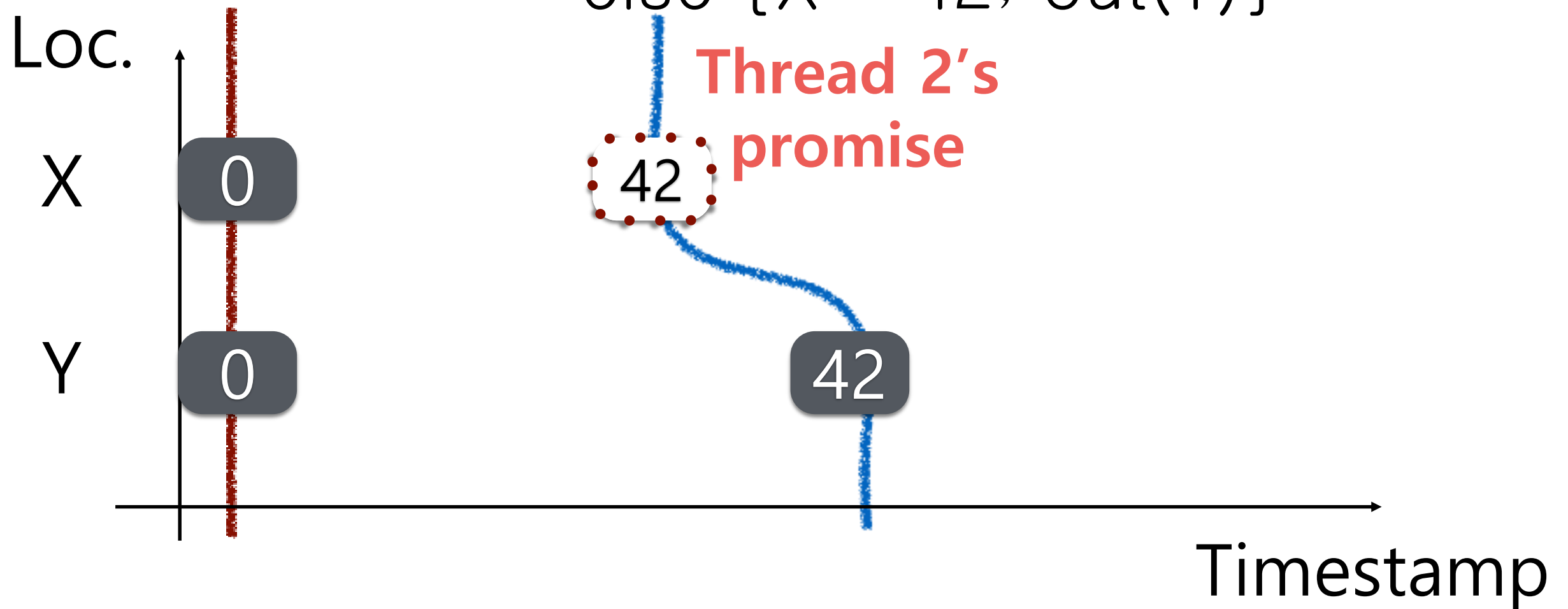
Thread 2 (allowed: $a=b=42, out(0)$)



$b = Y; c = X$

if ($b==42$) { $X = b; out(0)$ }

else { $X = 42; out(1)$ }



Example

Tweaked Store Hoisting

Thread 1

42

$a = X$

$Y = a$

Thread 2

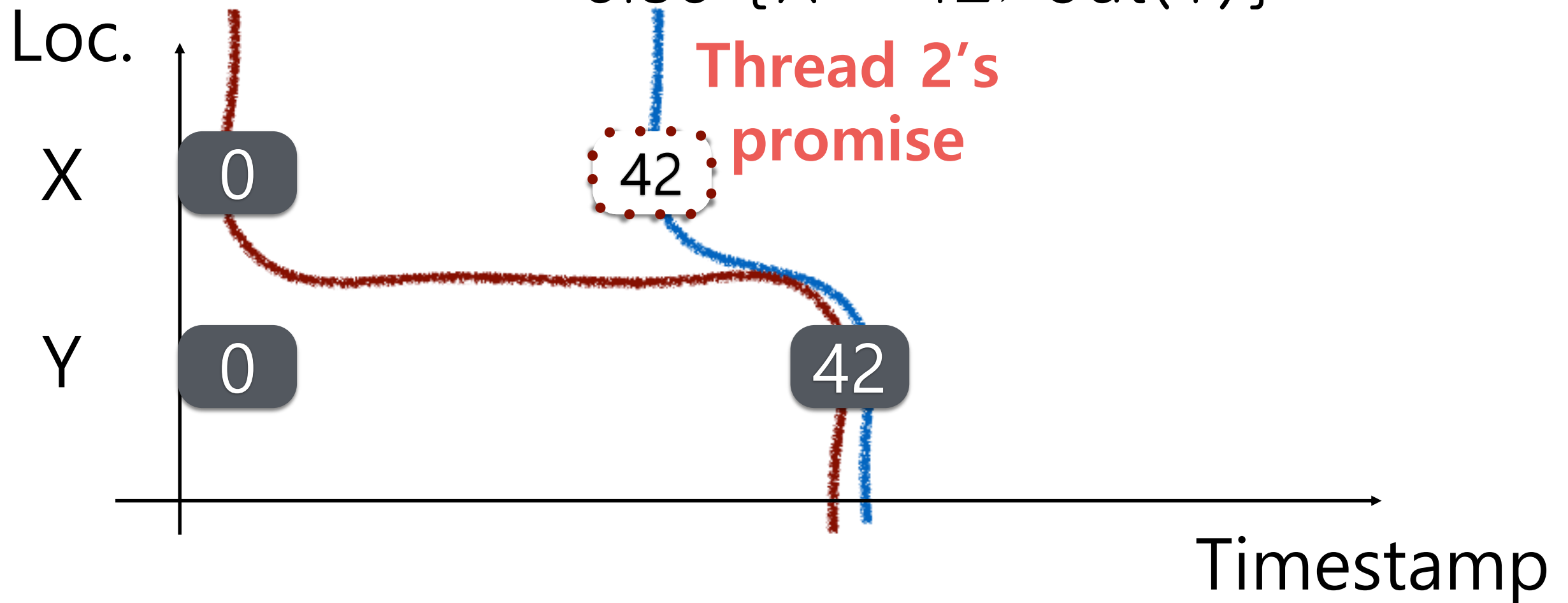
(allowed: $a=b=42, \text{out}(0)$)

42

$b = Y; c = X$

if ($b==42$) { $X = b; \text{out}(0)$ }

else { $X = 42; \text{out}(1)$ }



Example

Tweaked Store Hoisting

Thread 1

42

$a = X$

$Y = a$

Thread 2

42

0

$b = Y; c = X$

if ($b == 42$) { $X = b; \text{out}(0)$ }

else { $X = 42; \text{out}(1)$ }

(allowed: $a=b=42, \text{out}(0)$)

**Thread 2's
promise**



Example

Tweaked Store Hoisting

Thread 1

42

$a = X$

$Y = a$

Thread 2

42

0

$b = Y; c = X$

if ($b == 42$) { $X = b; \text{out}(0)$ }

else { $X = 42; \text{out}(1)$ }

(allowed: $a=b=42, \text{out}(0)$)

Loc.

X

0

42

Thread 2's
promise

Y

0

42

Timestamp

Example

Tweaked Store Hoisting

Thread 1

42

$a = X$

$Y = a$

Thread 2

42

0

(allowed: $a=b=42, out(0)$)

$b = Y; c = X$

if ($b==42$) { $X = b; out(0)$ }

else { $X = 42; out(1)$ }

Loc.

X

0

Y

0

**Thread 2's
promise**

42

42

Timestamp

Example Tweaked Store

False dependency
Causes no trouble!

Thread 1

42
a = X
Y = a

Thread 2

(allowed: a=b=42, out(0))

42 0
b = Y; c = X
if (b==42) {X = b; out(0)}
else {X = 42; out(1)}

Loc.

X

0

Y

0

Thread 2's
promise

42

42

Timestamp

Example

Tweaked Store Hoisting

Thread 1

42
 $a = X$
 $Y = a$

Thread 2 (allowed: $a=b=42, out(0)$)

42
 $b = Y; c = X$
if ($b==42$) { $X = b; out(0)$ }
else { $X = 42; out(1)$ }



Example

Tweaked Store Hoisting

Thread 1

42

$a = X$

$Y = a$

Thread 2

42

42

(allowed: $a=b=42, out(0)$)

$b = Y; c = X$

if ($b==42$) { $X = b; out(0)$ }

else { $X = 42; out(1)$ }

Loc.

X

0

Y

0

**Thread 2's
promise**

42

42

Timestamp

Example Tweaked Store

The step is
Not Certified!

Thread 1

42

$a = X$

$Y = a$

Thread 2

42

42

$b = Y; c = X$

if ($b == 42$) { $X = b; \text{out}(0)$ }

else { $X = 42; \text{out}(1)$ }

(allowed: $a=b=42, \text{out}(0)$)

Loc.

X

0

Y

0

42

**Thread 2's
promise**

42

Timestamp

Example

Classical Out-of-thin-air (OOTA)

Thread 1



$a = X$

$Y = a$

Thread 2



$b = Y$

$X = b$

(forbidden: $a=b=42$)

Loc.

X

0

Y

0

Timestamp

Example

Classical Out-of-thin-air (OOTA)

Thread 1



$a = X$

$Y = a$

Thread 2



$b = Y$

$X = b$

(forbidden: $a=b=42$)

Loc.

X

0

Y

0

Thread 2's

42

promise?

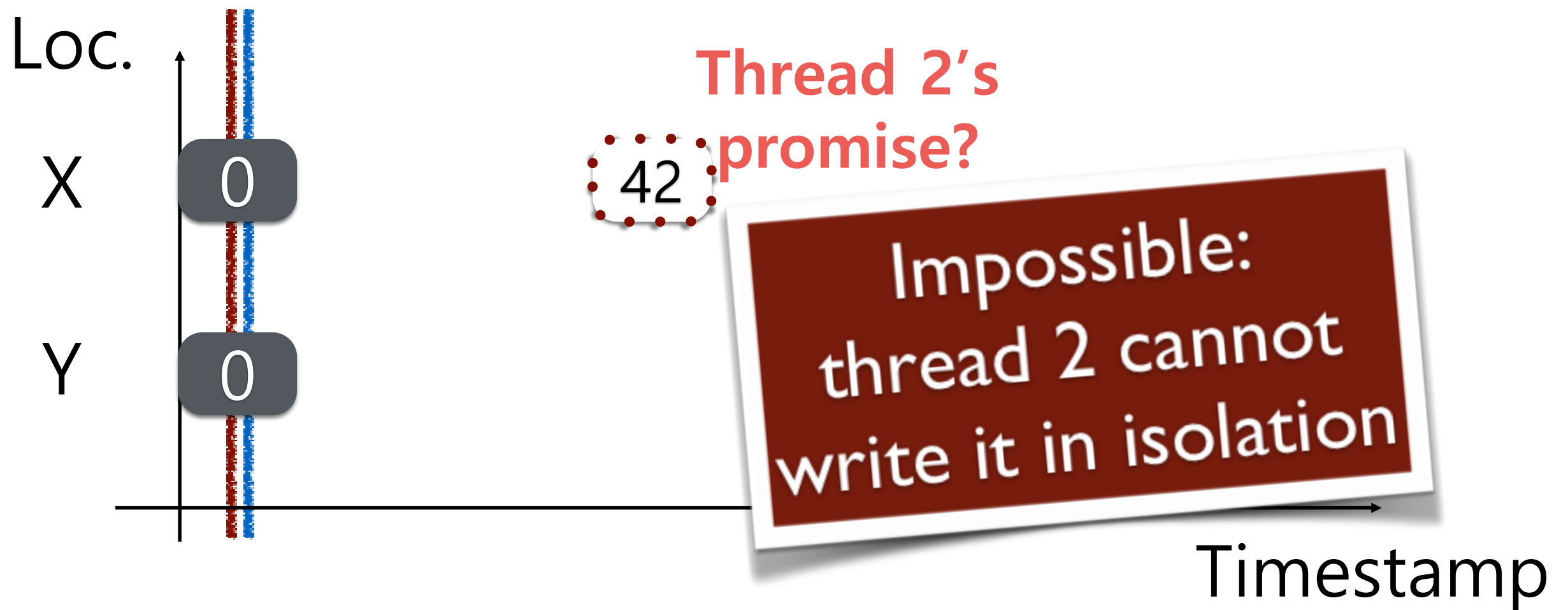
Timestamp

Example

Classical Out-of-thin-air (OOTA)

Thread 1
→
 $a = X$
 $Y = a$

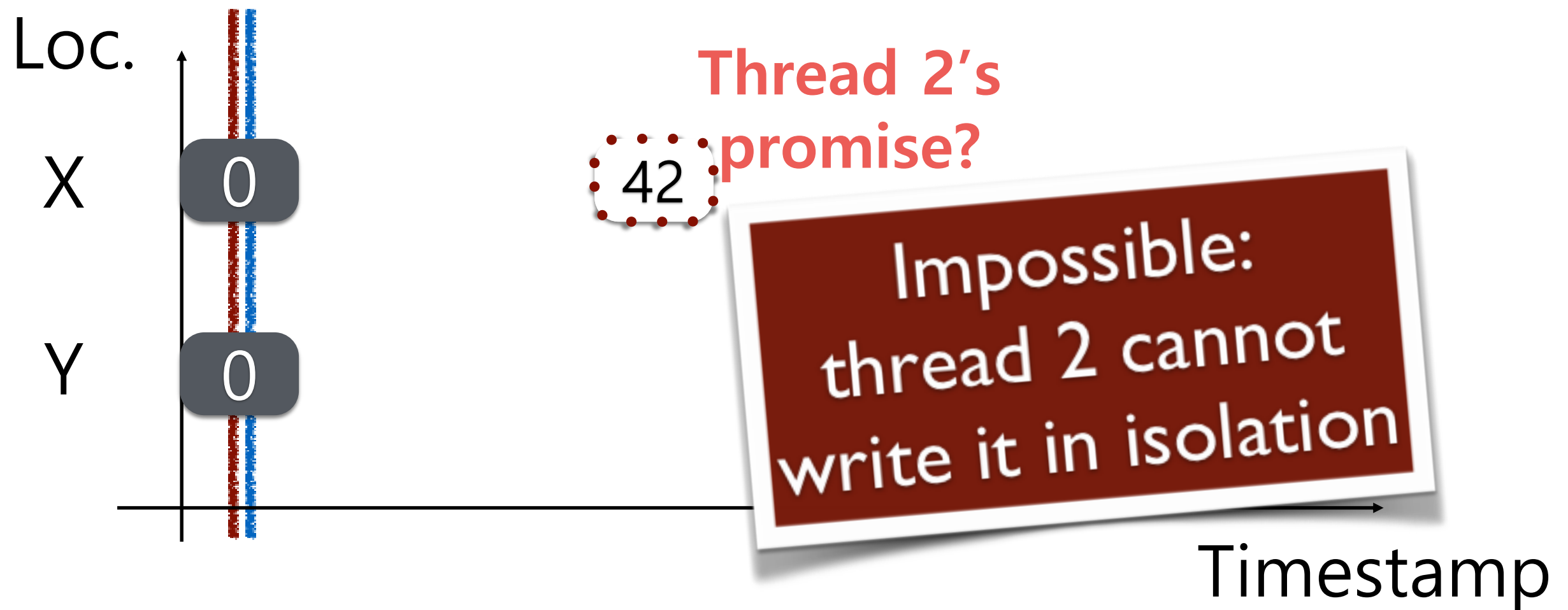
Thread 2
→
 $b = Y$
 $X = b$ (forbidden: $a=b=42$)



Example

Classical Out-of-thin-air (OOTA)

Promises: “Semantic Solution” to OOTA



The Message Passing Example

Thread 1



`D = 42`

`F = 1`

Thread 2



```
while (1) {  
    f = F  
    if (f) break  
}  
d = D
```



The Message Passing Example

Thread 1



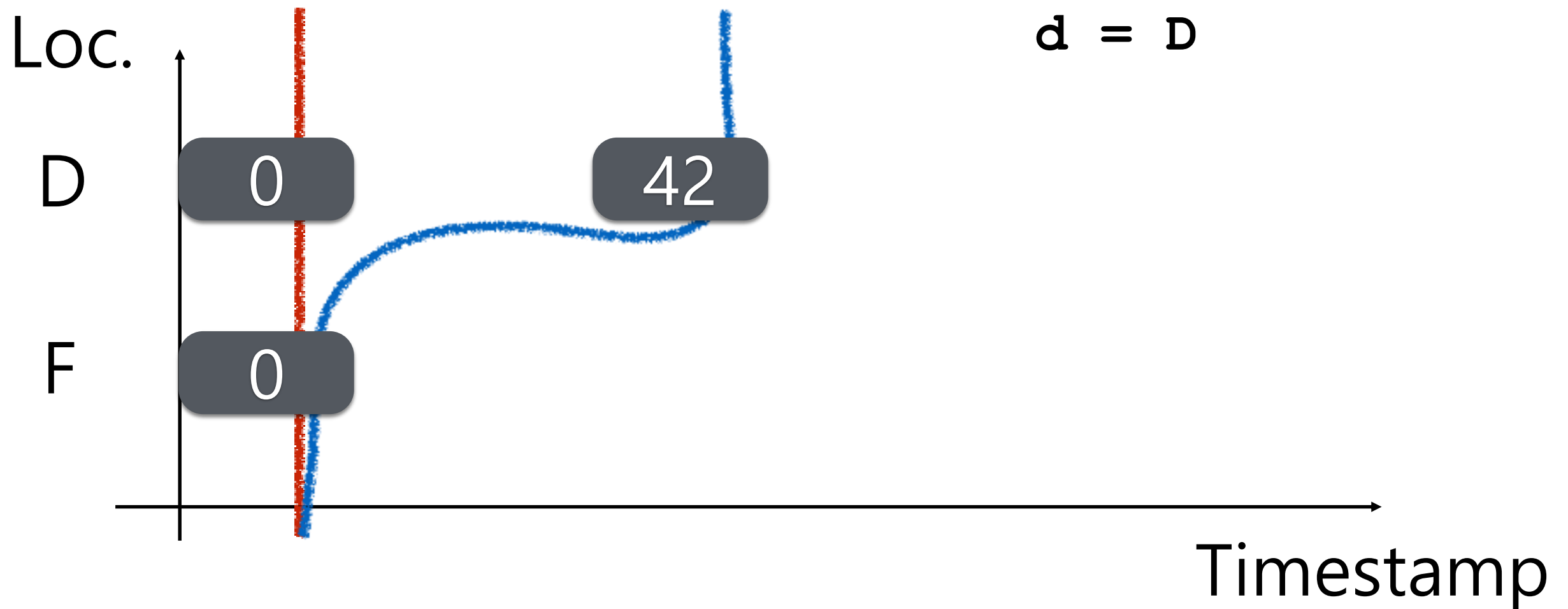
`D = 42`

`F = 1`

Thread 2



```
while (1) {  
    f = F  
    if (f) break  
}  
d = D
```



The Message Passing Example

Thread 1

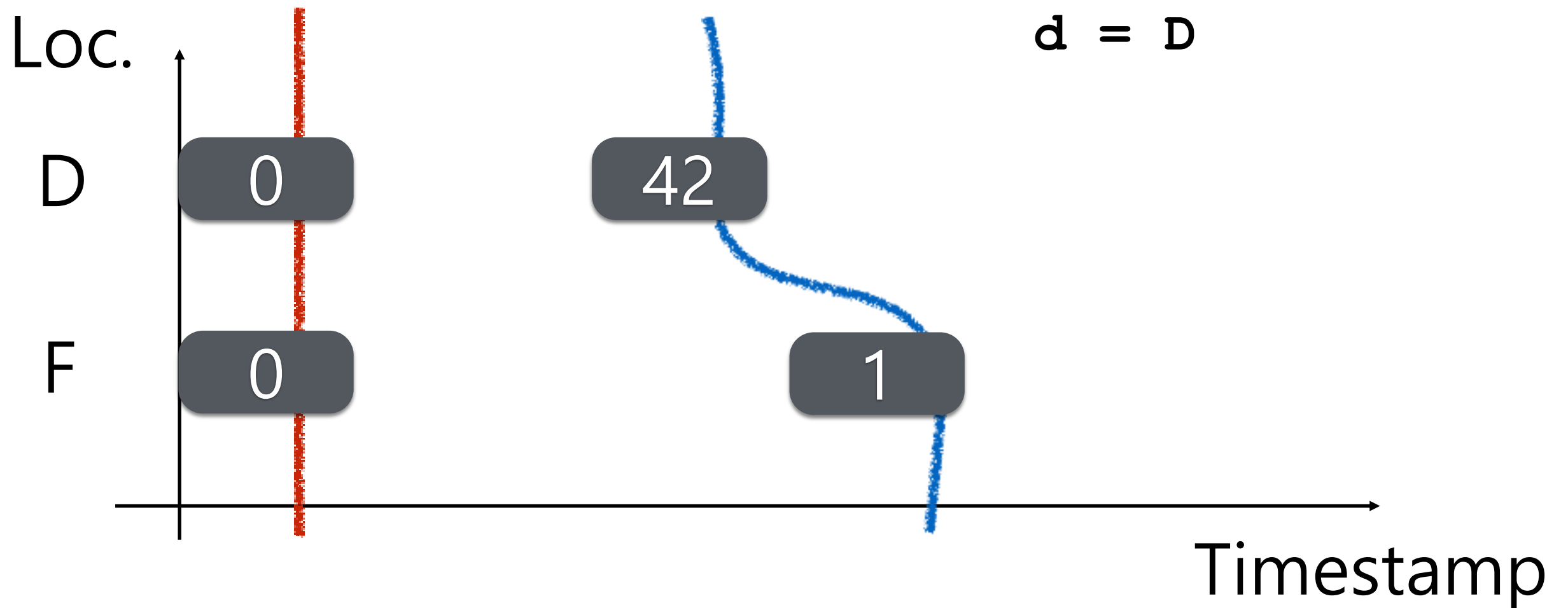


`D = 42`
`F = 1`

Thread 2



```
while (1) {  
    f = F  
    if (f) break  
}  
d = D
```



The Message Passing Example

Thread 1

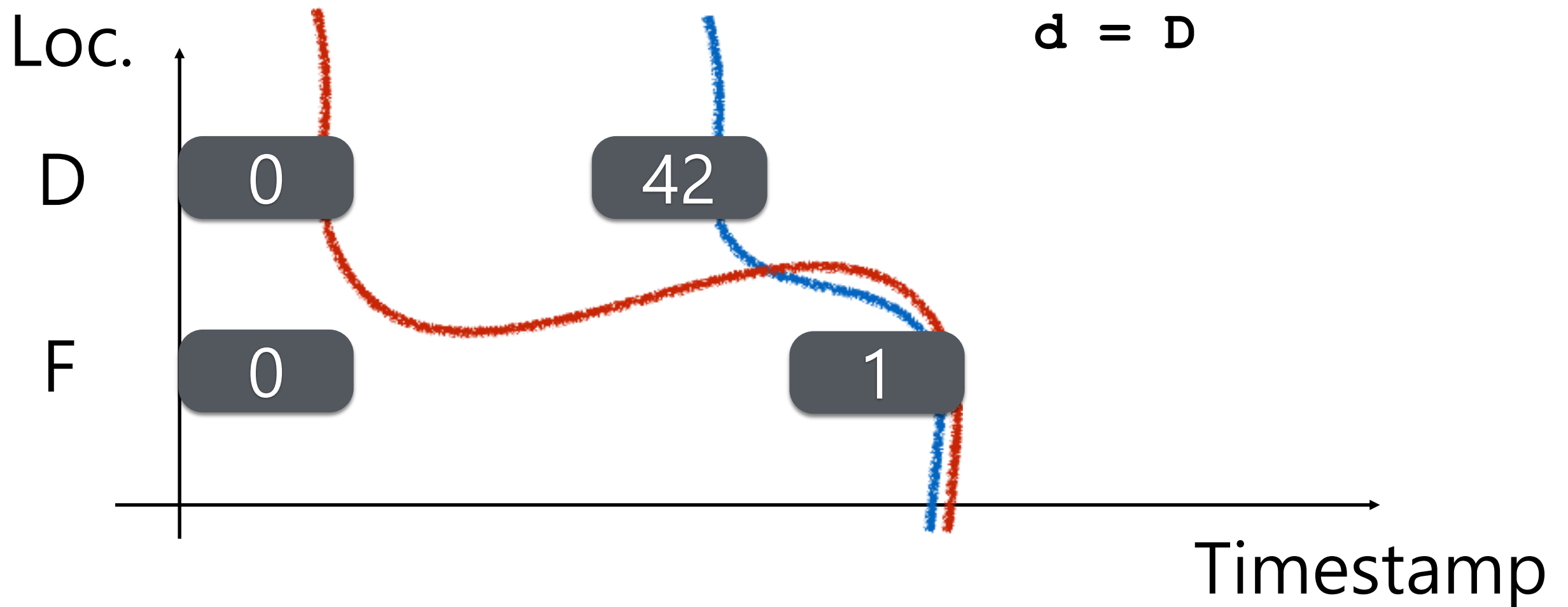


`D = 42`
`F = 1`

Thread 2



```
while (1) {  
    f = F  
    if (f) break  
}  
d = D
```



The Message Passing Example

Thread 1



`D = 42`

`F = 1`

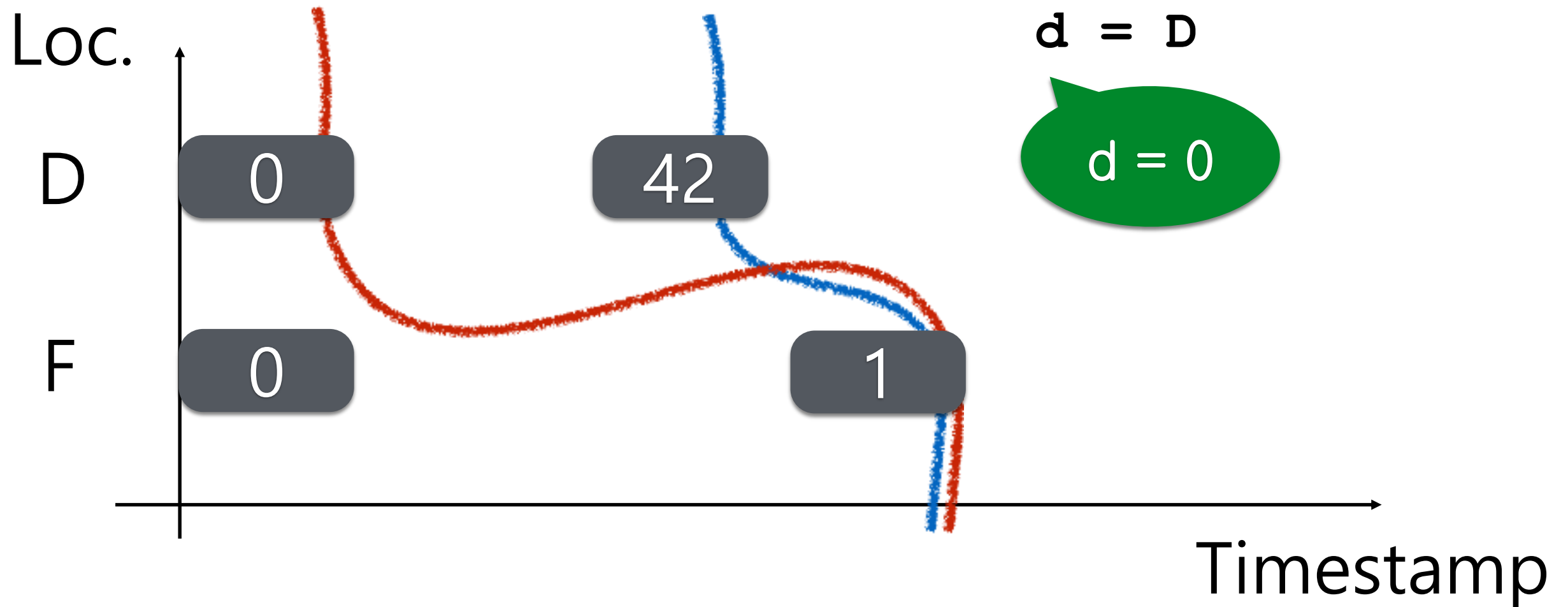
Thread 2



```
while (1) {  
    f = F  
    if (f) break  
}
```

`d = D`

`d = 0`



Release & Acquire

Thread 1



D = 42

F = 1 [rel]

Thread 2



```
while (1) {  
    f = F [acq]  
    if (f) break  
}  
d = D
```



Release & Acquire

Thread 1



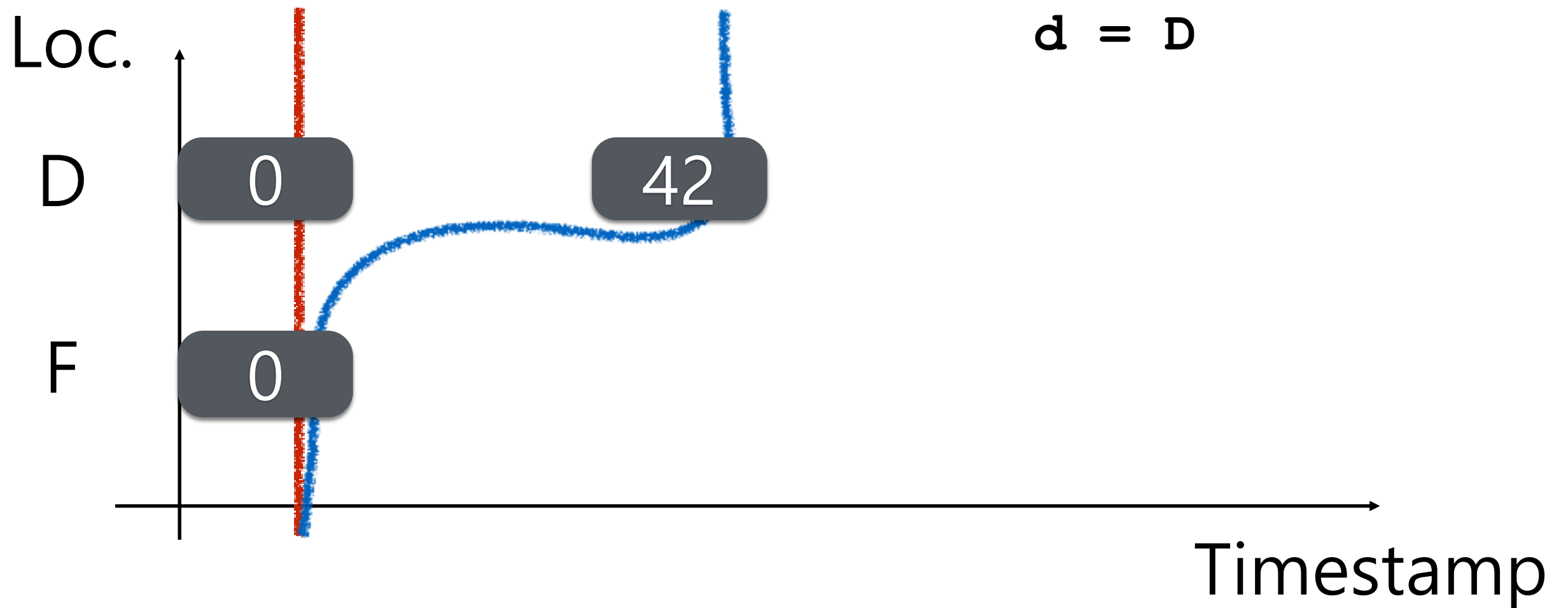
D = 42

F = 1 [rel]

Thread 2



```
while (1) {  
    f = F [acq]  
    if (f) break  
}  
d = D
```



Release & Acquire

Thread 1



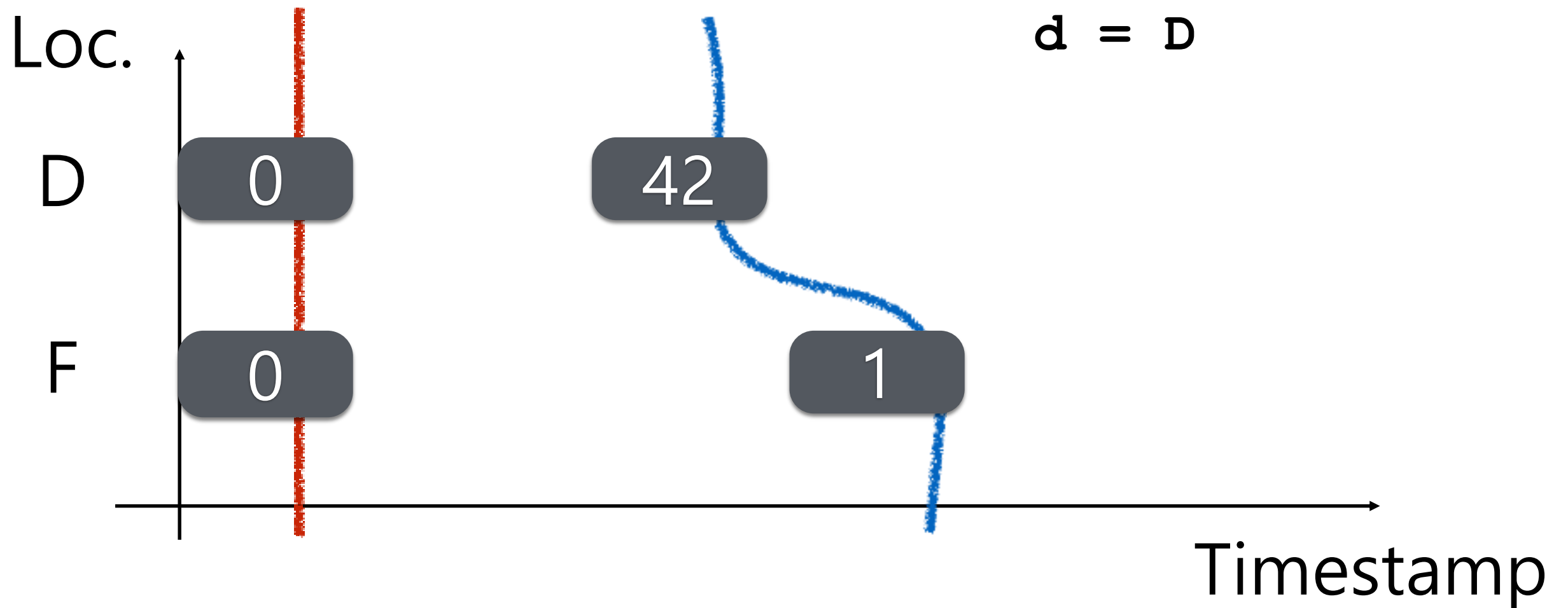
D = 42

F = 1 [rel]

Thread 2



```
while (1) {  
    f = F [acq]  
    if (f) break  
}  
d = D
```



Release & Acquire

Thread 1



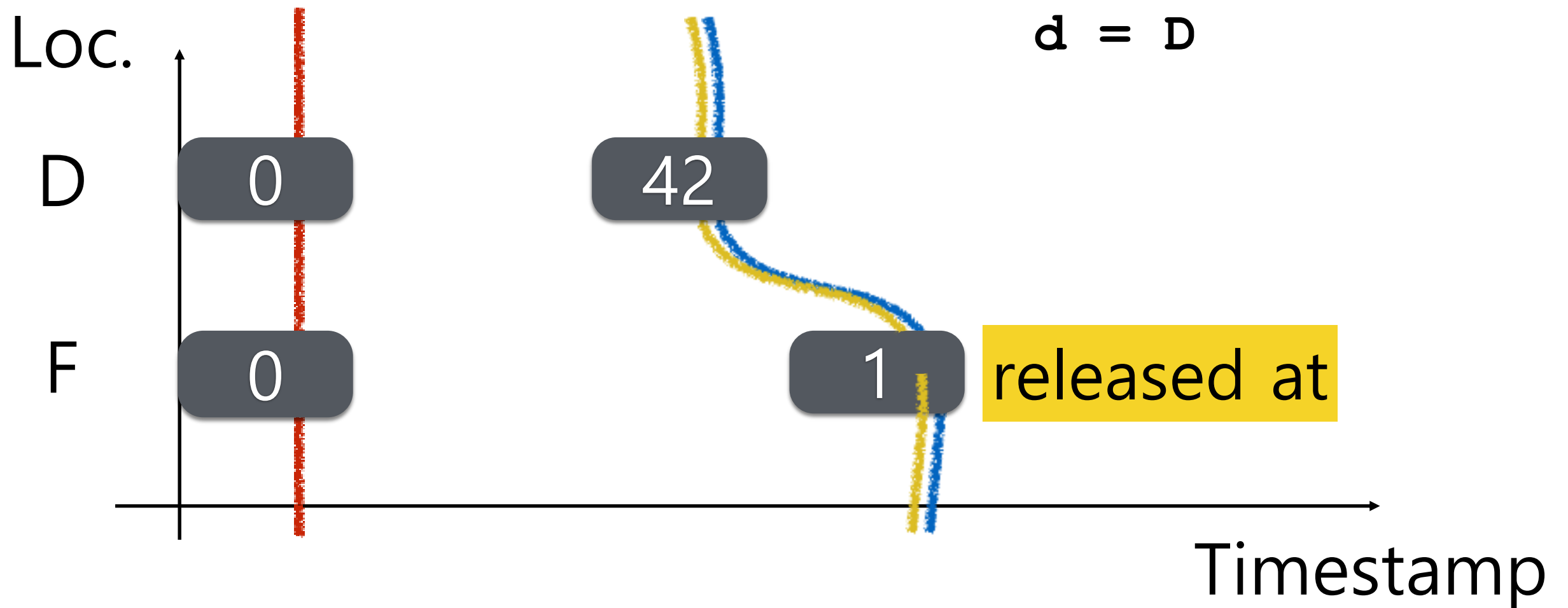
D = 42

F = 1 [rel]

Thread 2



```
while (1) {  
    f = F [acq]  
    if (f) break  
}  
d = D
```



Release & Acquire

Thread 1



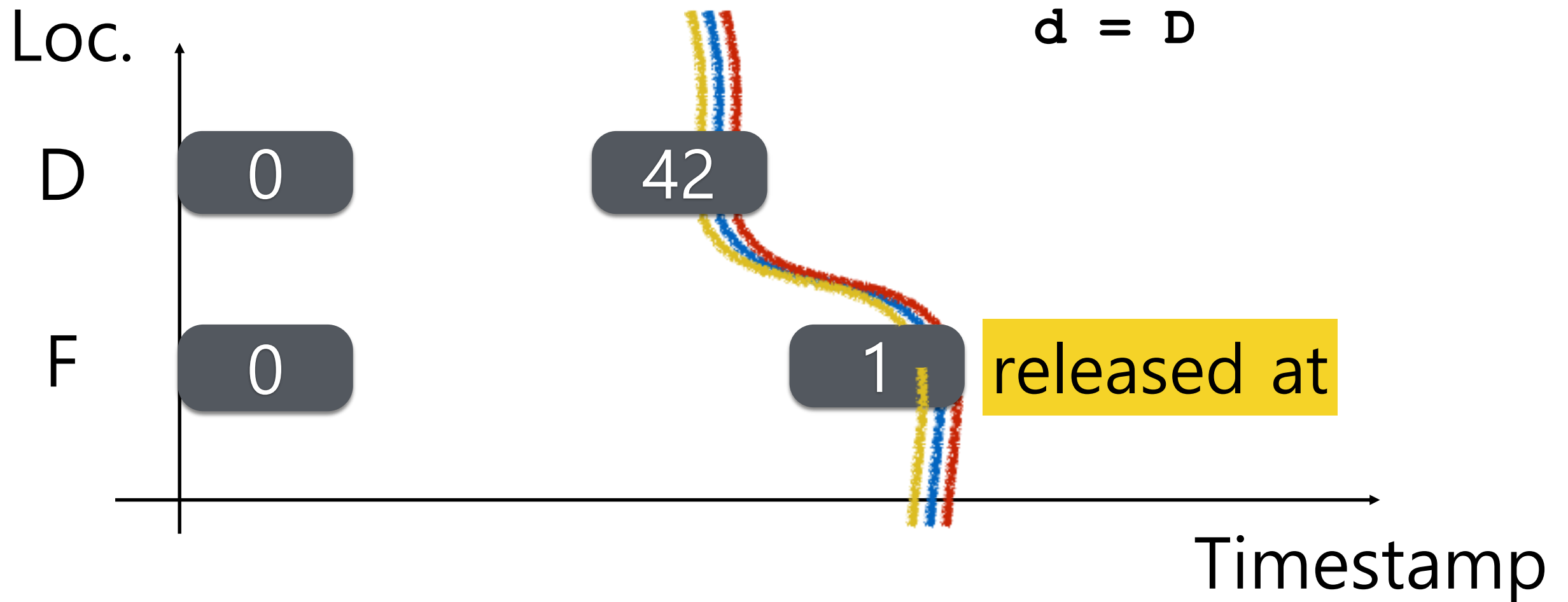
D = 42

F = 1 [rel]

Thread 2



```
while (1) {  
    f = F [acq]  
    if (f) break  
}  
d = D
```



Release & Acquire

Thread 1



D = 42

F = 1 [rel]

Thread 2



```
while (1) {  
    f = F [acq]  
    if (f) break  
}
```

d = D

d = 42

Loc.

D

0

42

F

0

1

released at

Timestamp

Results (1/2)

Compiler/HW Optimizations

- **Operational semantics** for C/C++ concurrency: plain/relaxed/release/acquire r/w/u/fence, SC fence
- **Compiler optimizations**  (reordering, merge, dead load elim., ...)
- **Compilation** to x86  & Power 

Results (2/2)

Reasoning Principles

- **DRF-SC: Data Race Freedom $\Rightarrow$ SC** 
 - DRF-PromiseFree:
DRF $\Rightarrow$ semantics w/o promises 
- **Invariant-based logic:** 
soundness of global invariant (e.g. $a=b=X=Y=0$)
- <http://sf.snu.ac.kr/promise-concurrency>



More comprehensive semantics for C/C++ concurrency

- **DRF-SC: Data Race Freedom $\Rightarrow$ SC** 
 - DRF-PromiseFree:
DRF $\Rightarrow$ semantics w/o promises 
- **Invariant-based logic:** 
soundness of global invariant (e.g. $a=b=X=Y=0$)
- <http://sf.snu.ac.kr/promise-concurrency>



Future Work

- Supporting **SC reads & writes**
(We found a **flaw in C/C++11 on SC**)
- Supporting **consume** reads
- Developing a rich **program logic** &
Verifying fine-grained concurrent programs